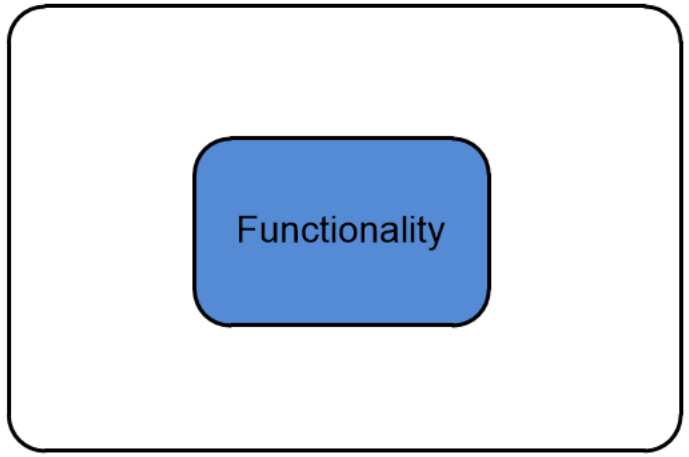
Enterprise System Integration (MTAT.03.229)

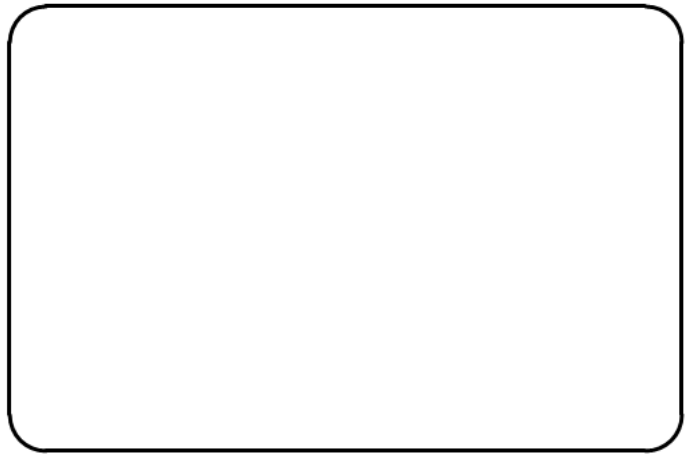
LECTURE2: WEB SERVICES – REST, AND RESTFUL API – CRUD OPERATIONS - I

MOHAMAD GHARIB

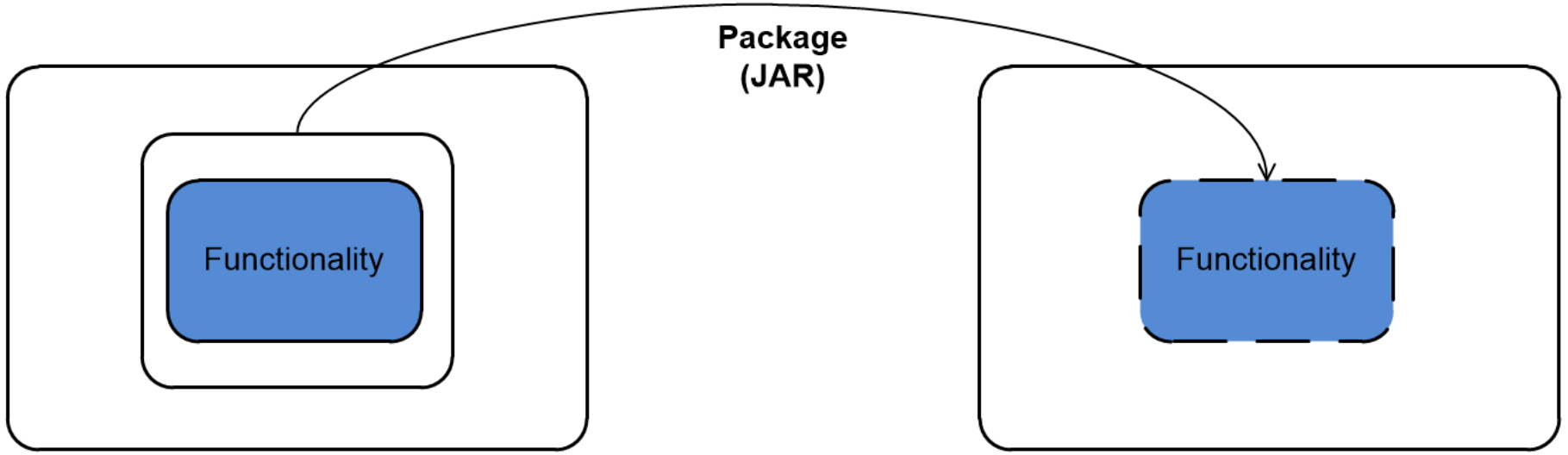
UNIVERSITY OF TARTU

Distributed Computing Technologies

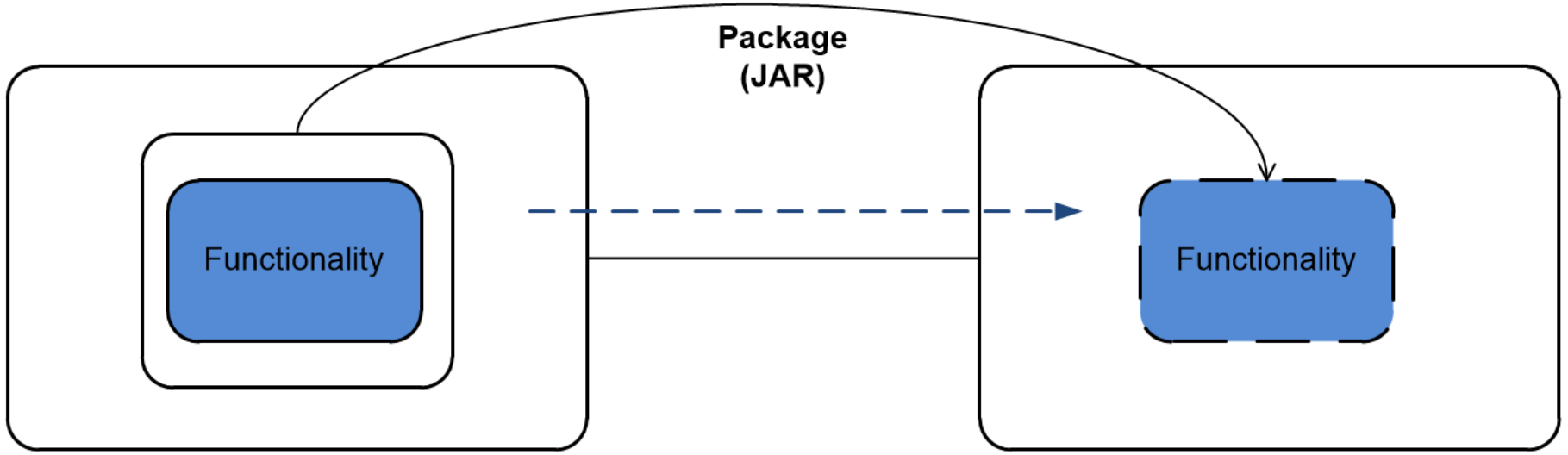




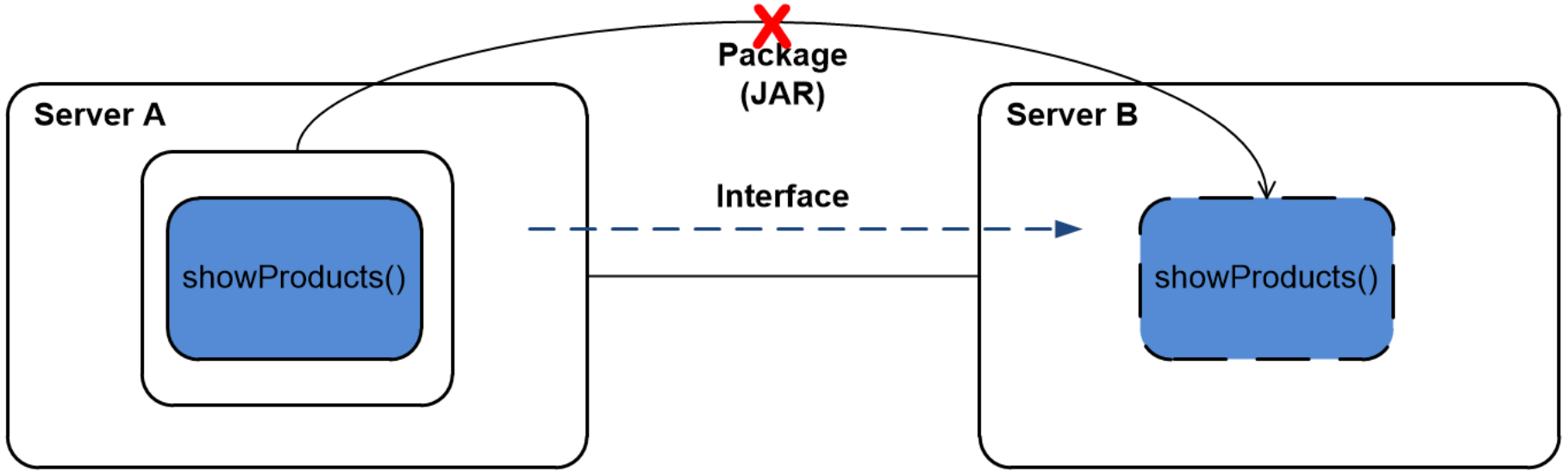
Distributed Computing Technologies



Distributed Computing Technologies



Distributed Computing Technologies



Distributed Computing – brief history

Java RMI (Remote Method Invocation) performs remote method invocation with support for direct transfer of serialized Java classes and distributed garbage-collection.

Common Object Request Broker Architecture (CORBA) – OMG: CORBA is language- and OS-independence, freedom from technology-linked implementations, and freedom from the details of distributed data transfers. ***CORBA*** specification is complex, expensive to implement entirely, and often ambiguous. Also, it faced problems with Firewalls.

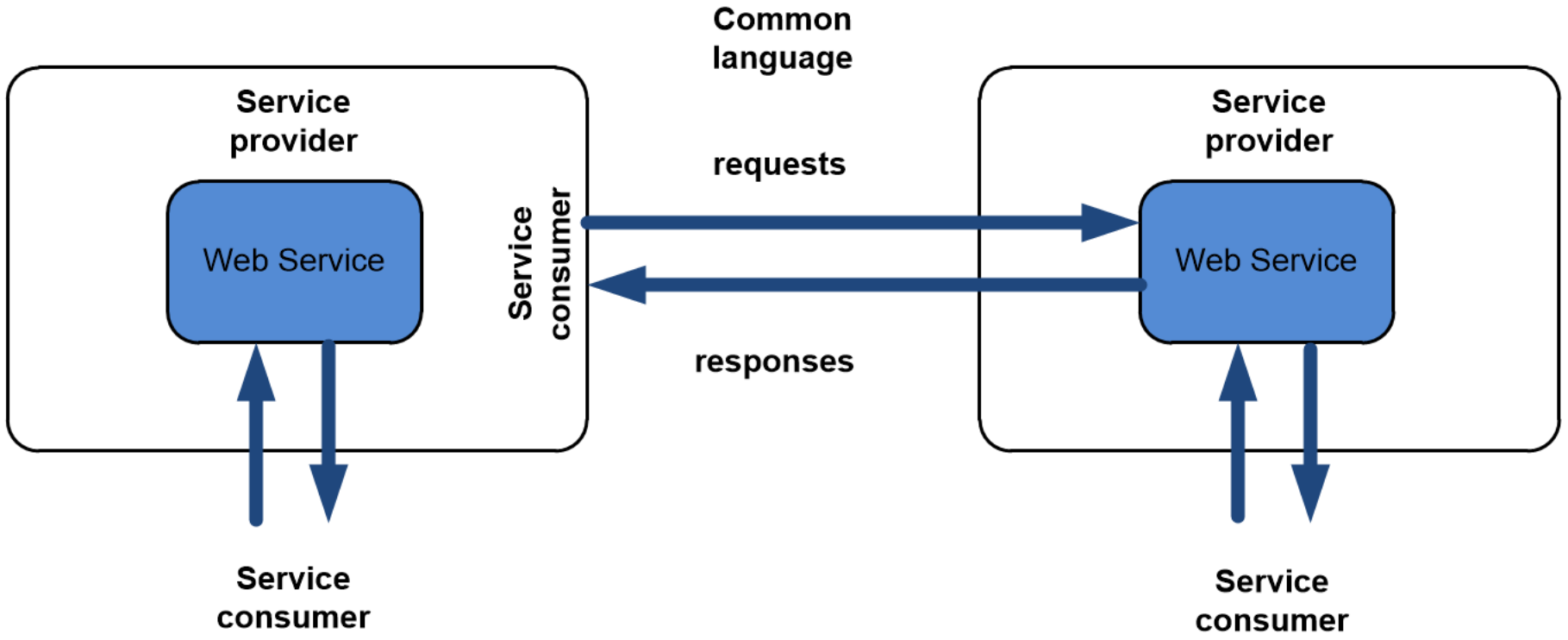
Web Services (W3C) provide a standard means of interoperating between different software applications, running on various platforms and/or frameworks[1].

[1]Ahalt, Mark D. Hanes Stanley C., C. Hanes Stanley, and Ashok K. Krishnamurthy. "A Comparison of Java RMI, CORBA, and Web Services Technologies for Distributed SIP Applications." (2002).

Web Services

A Web Service is a ***standards-based***, ***language-agnostic*** software entity, which accepts ***well-defined*** formatted ***requests*** from other software entities, ***producing well-defined*** formatted ***responses***.

Web Services



A Web Service is a ***standards-based***, ***language-agnostic*** software entity, which accepts ***well-defined*** formatted ***requests*** from other software entities, ***producing well-defined*** formatted ***responses***.

Web Services advantages

Solve most communication problems, as interaction is built on standardized languages/protocols (e.g., XML, WSDL, HTTP).

Reuse, a service within the system is only ever coded once and used over and over again by various applications.

Interoperability, web service are not designed to work on a specific language nor platforms.

Integration, web services enable easier communications within and across organisations.

Composability, several web *services can be composed to deliver services than none of these services can be provide by its own.*

Web Services key features

Discoverability, web services need to be "effectively" discovered to allow other software entities to consume them.

Abstraction, web *services hide their logic from service consumers, they share only what is required to deliver the service.*

Autonomy, a web *service* has almost complete control over the logic it encapsulates.

Loosely coupled, each service exists independently of the other services that make up the application.

Statelessness, stateless services treat each request separately regardless of any previous interaction, which is very important for providing scalable services[2].

[2]Stateful vs Stateless Architecture: Why Stateless Won - https://www.virtasant.com/blog/stateful-vs-stateless-architecture-why-stateless-won

Web Services

Web services:

SOAP (Simple Object Access Protocol) - JAX-WS (Java API for XML Web Services) is a standardized API for creating and consuming SOAP.

REST (Representational state transfer) - JAX-RS (Java API for RESTful Web Services) is an architecture that provides support in creating web services according to REST.

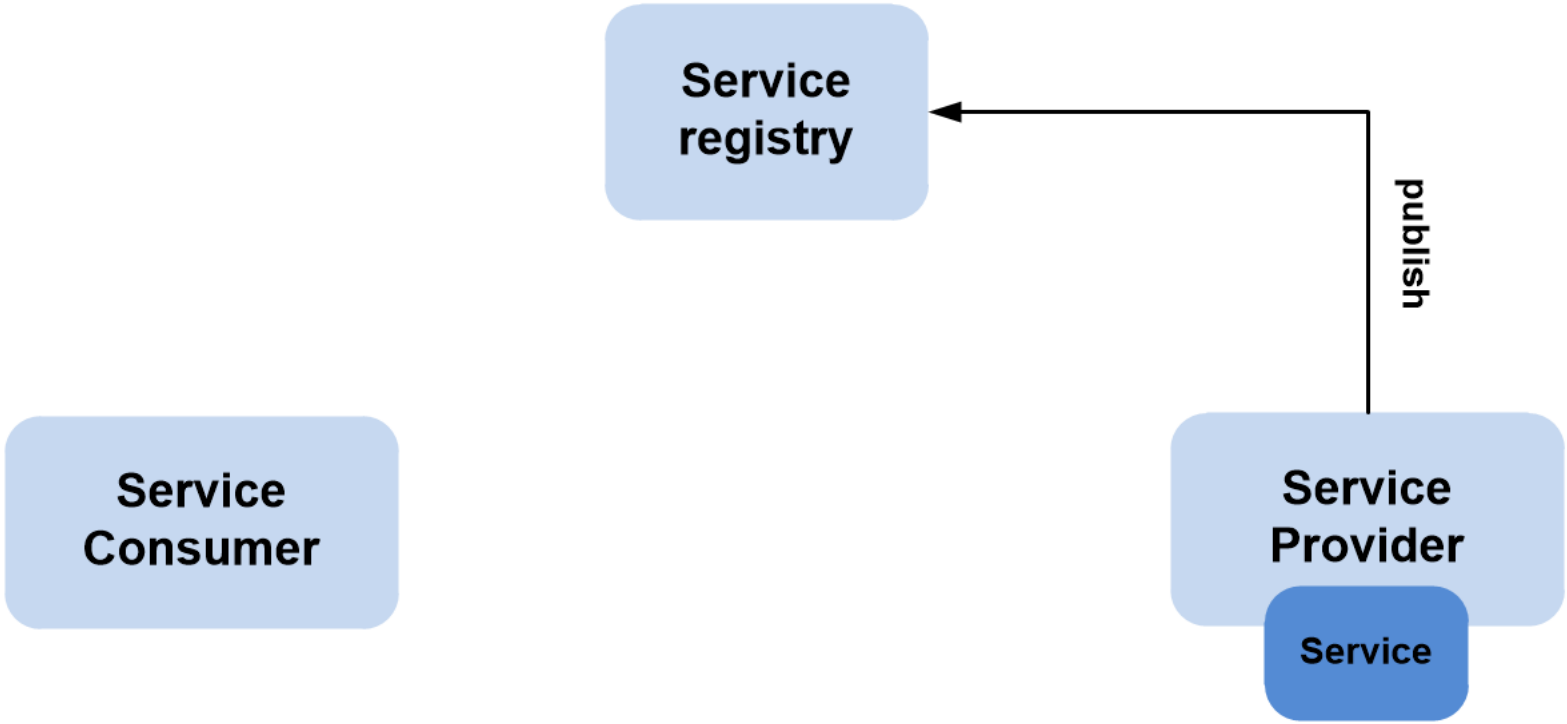
SOAP

SOAP architecture

SOAP is an ***XML-based protocol*** for accessing web services over HTTP.

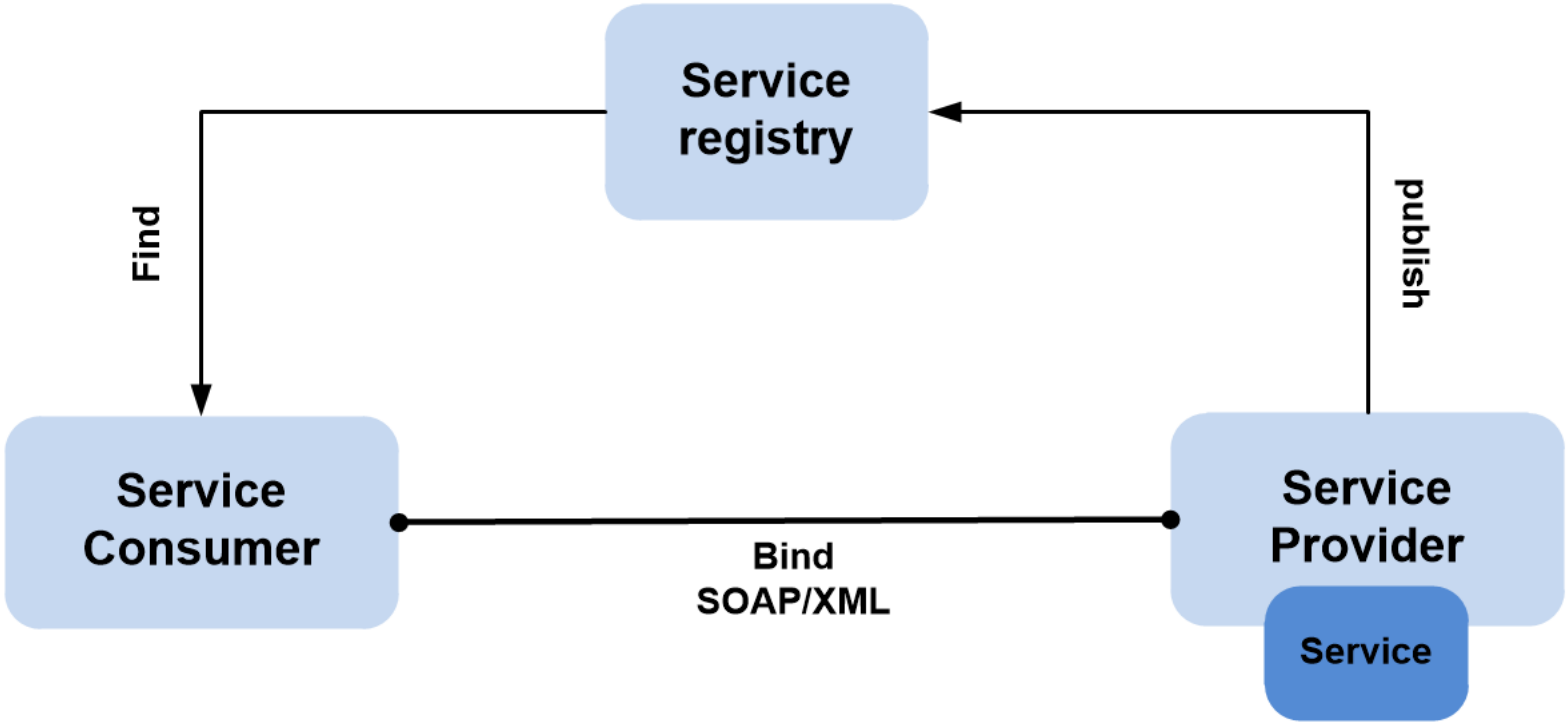
The ***SOAP architecture*** contains three key entities: 1- ***Service provider***, 2- ***Service requester***, and 3- ***Service registry***.

SOAP architecture



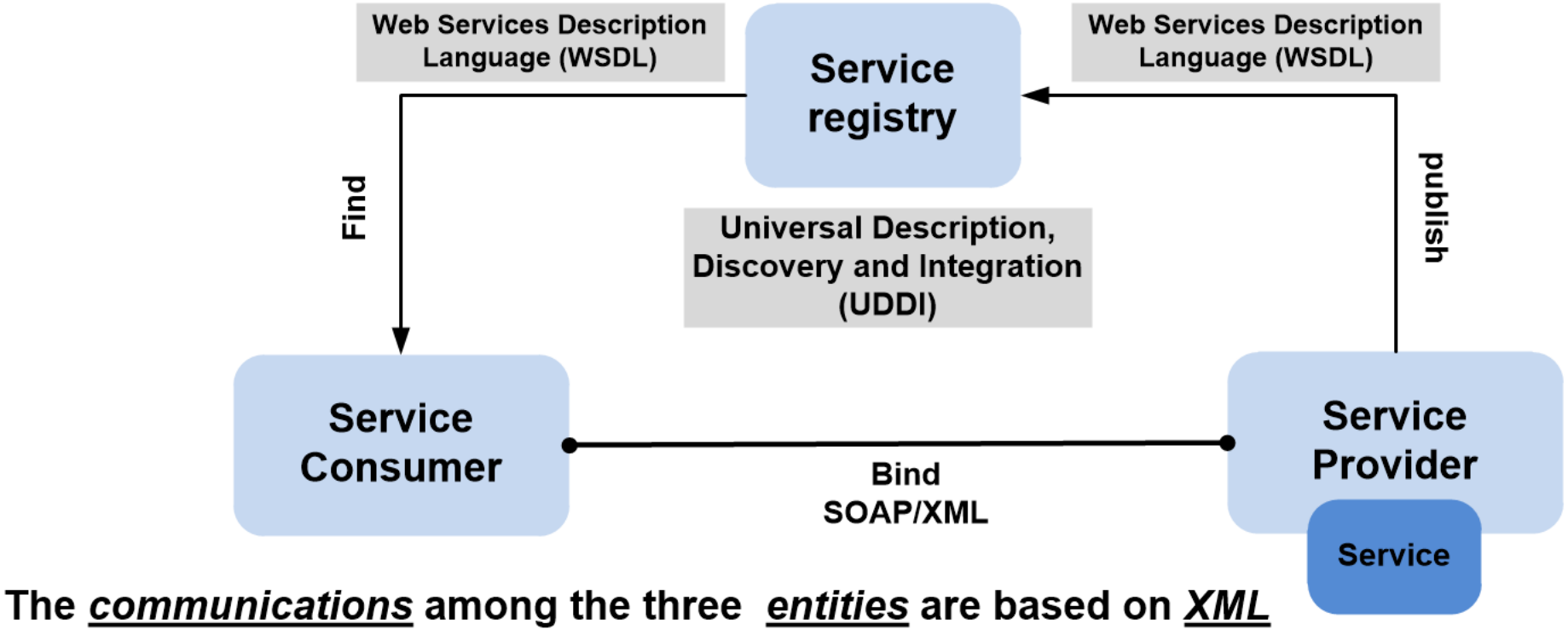
The ***service provider*** ***creates*** a ***SOAP service*** and ***publishes*** its ***description*** in the ***service registry***.

SOAP architecture



The ***service requester query*** the ***service registry*** to finds the ***service*** , retrieves the ***service description***, and then uses the description to ***bind*** to the ***service implementation*** and start ***using*** it.

SOAP architecture

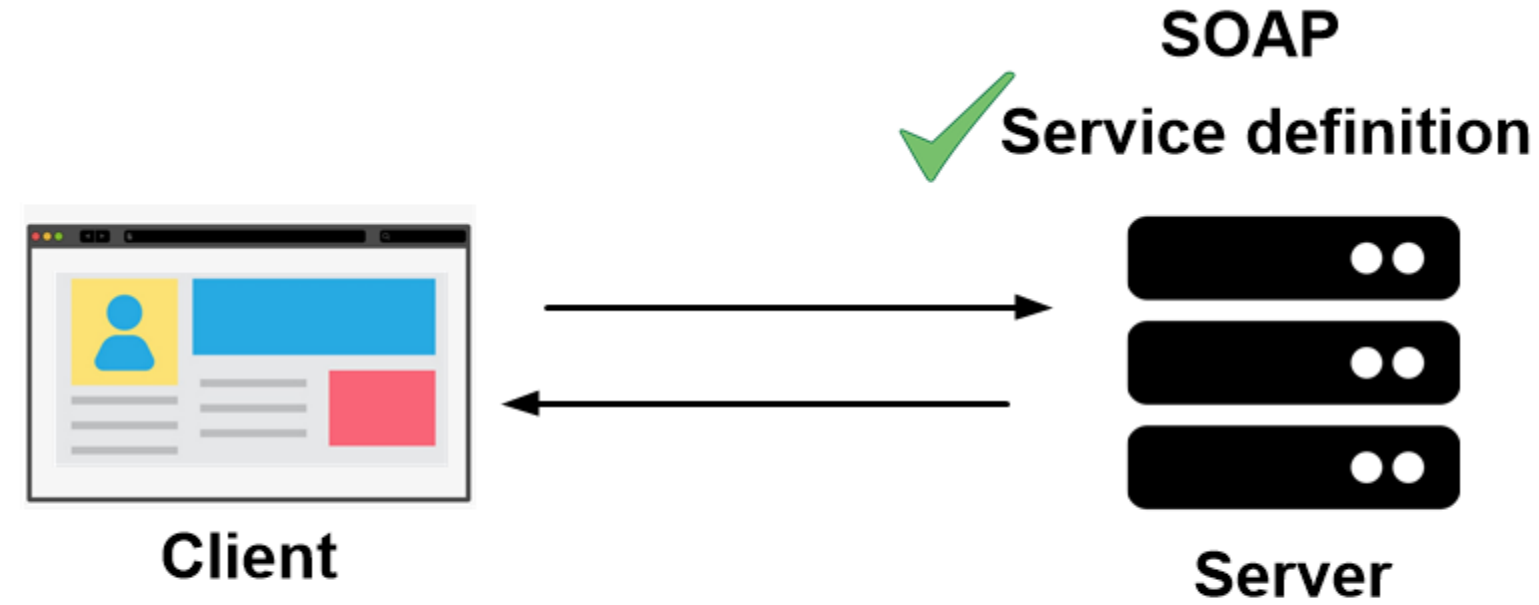


WSDL provides information on how to ***use*** a ***web service***, including a description of the service operations and ***binding*** information.

UDDI defines a set of APIs for both service publication and discovery.

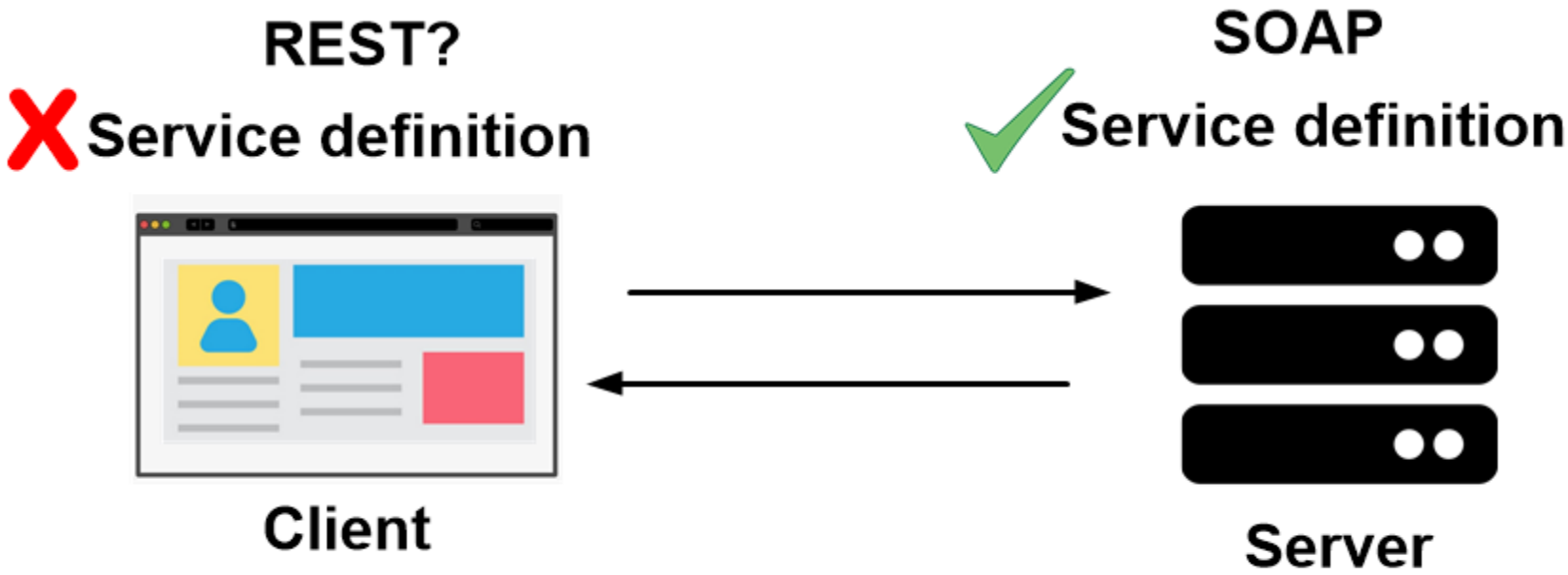
REST

REST - Representational State Transfer



SOAP protocol governs the interaction between the service provider and service consumer

REST - Representational State Transfer

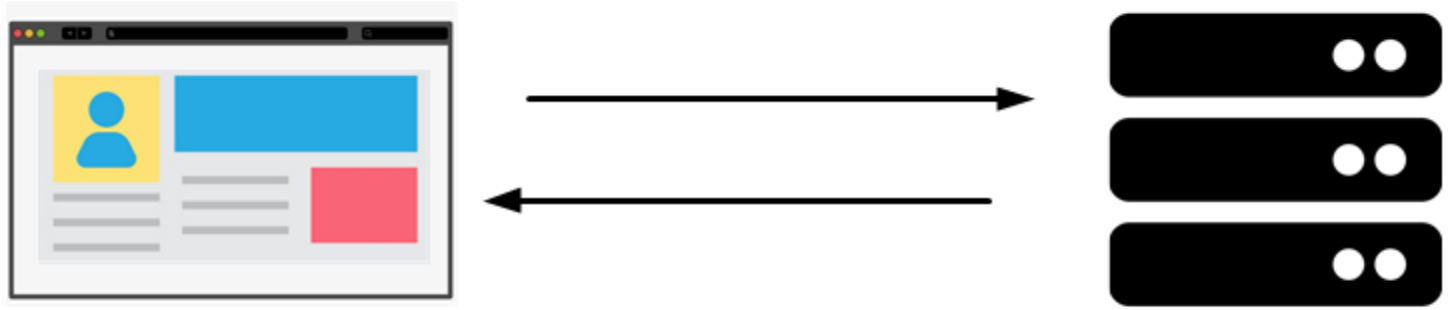


SOAP protocol governs the interaction between the service provider and service consumer

REST - Representational State Transfer

REST?

✗ Service definition



Client

Server

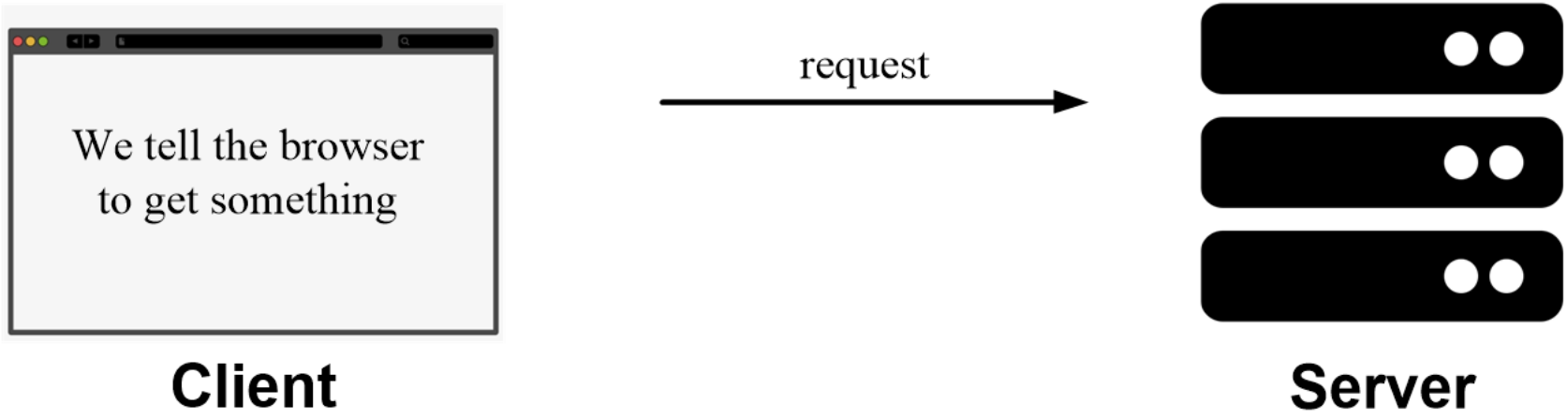
How does REST works?

How the front-end works

How the front-end works

scheme://domain:port/path?query#fragment_id

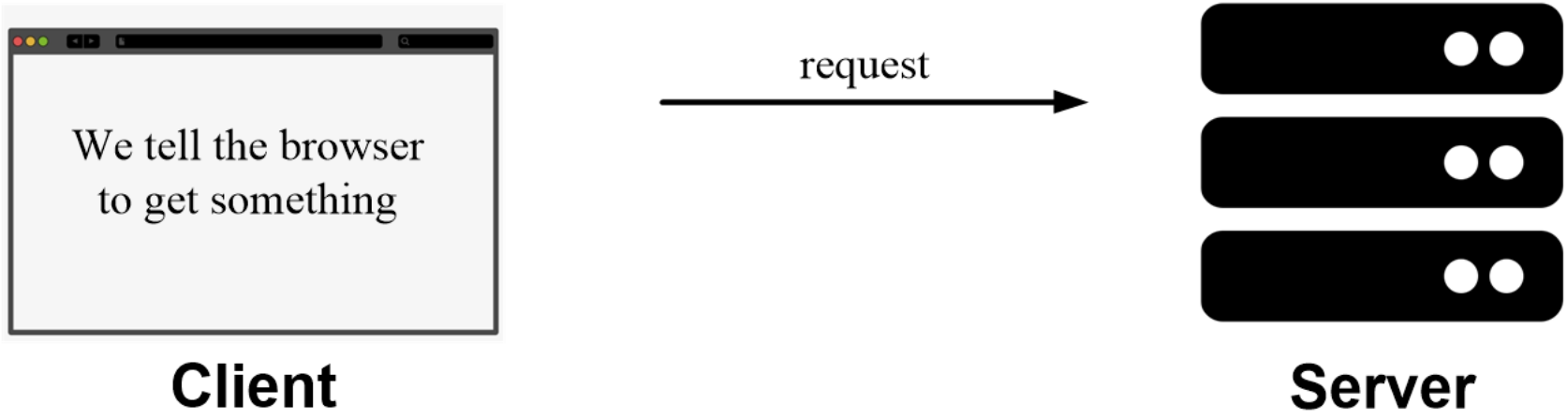
http://www.example.com/index.html



How the front-end works

scheme://domain:port/path?query#fragment_id

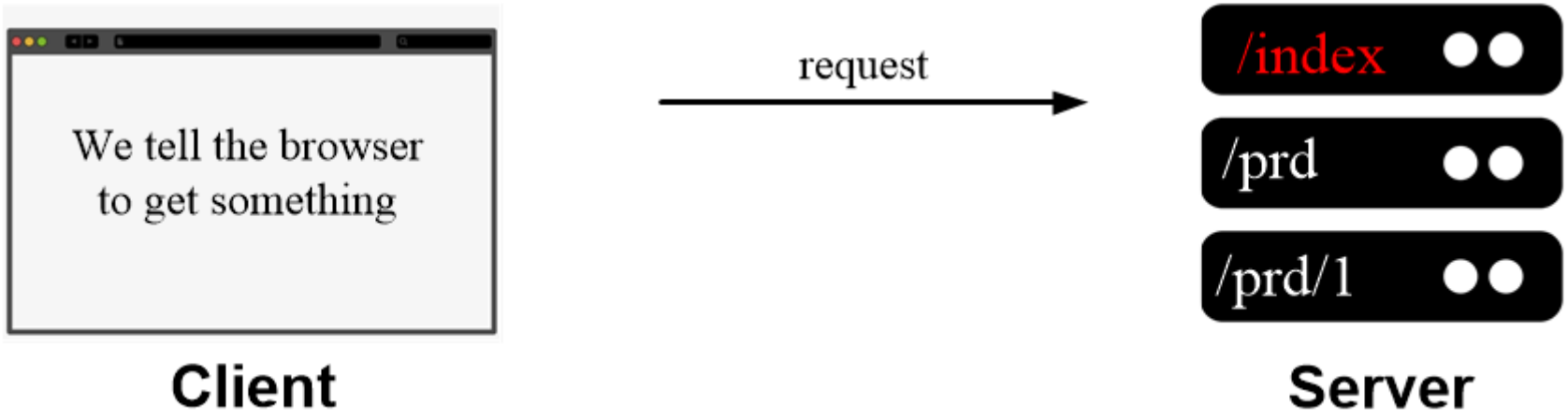
http://www.example.com/index.html



How the front-end works

scheme://domain:port/path?query#fragment_id

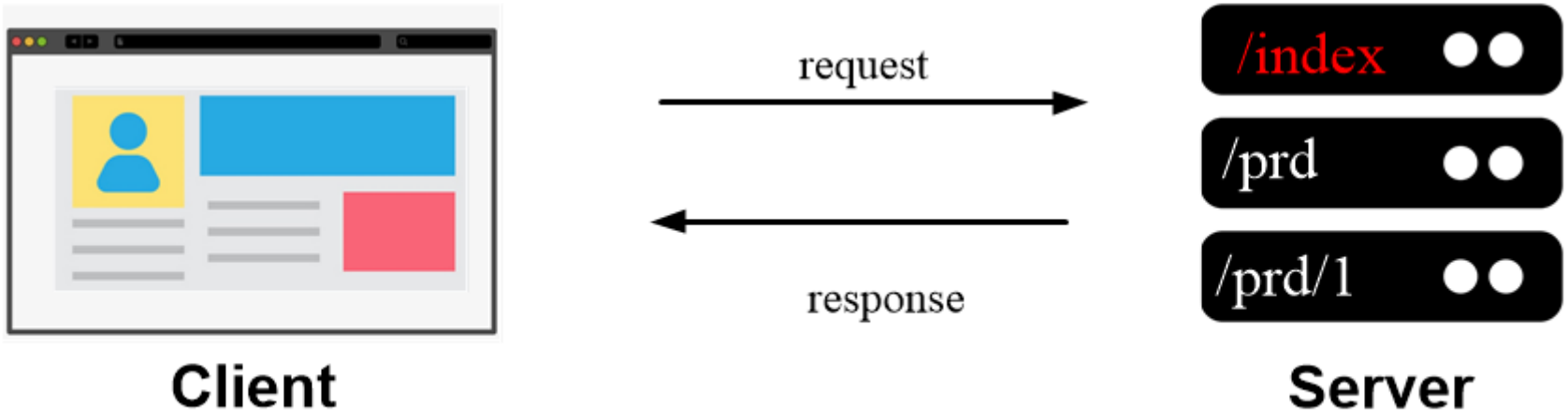
http://www.example.com/index.html



How the front-end works

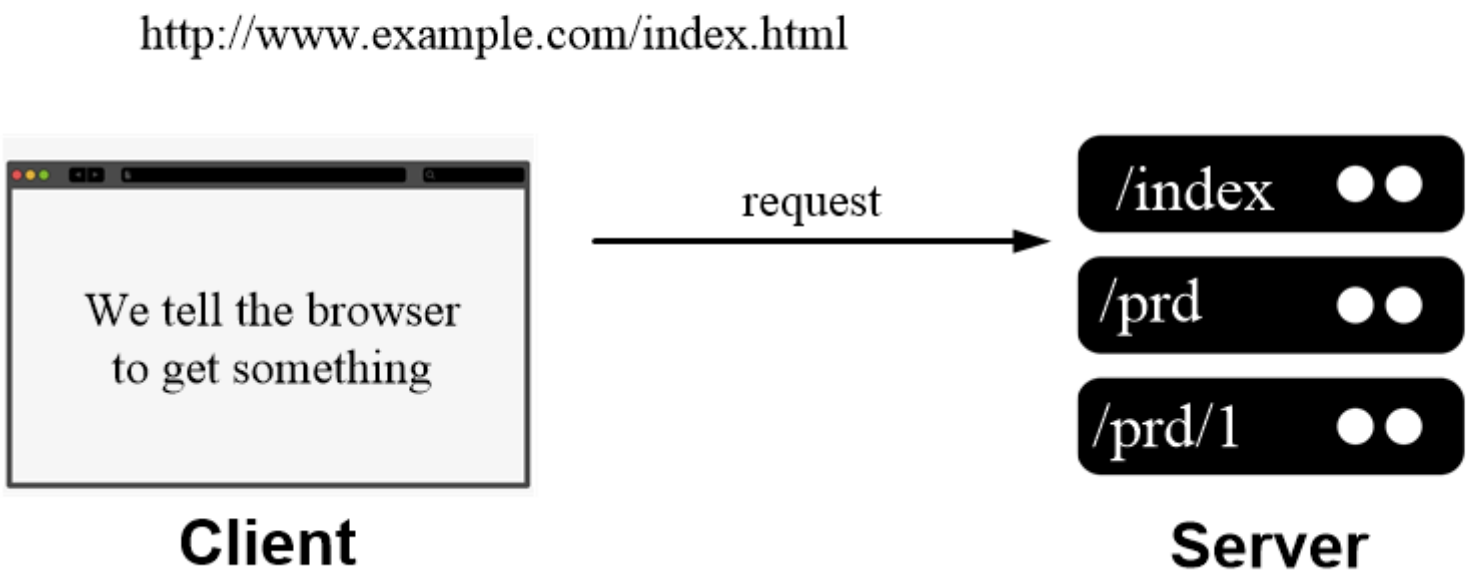
scheme://domain:port/path?query#fragment_id

http://www.example.com/index.html

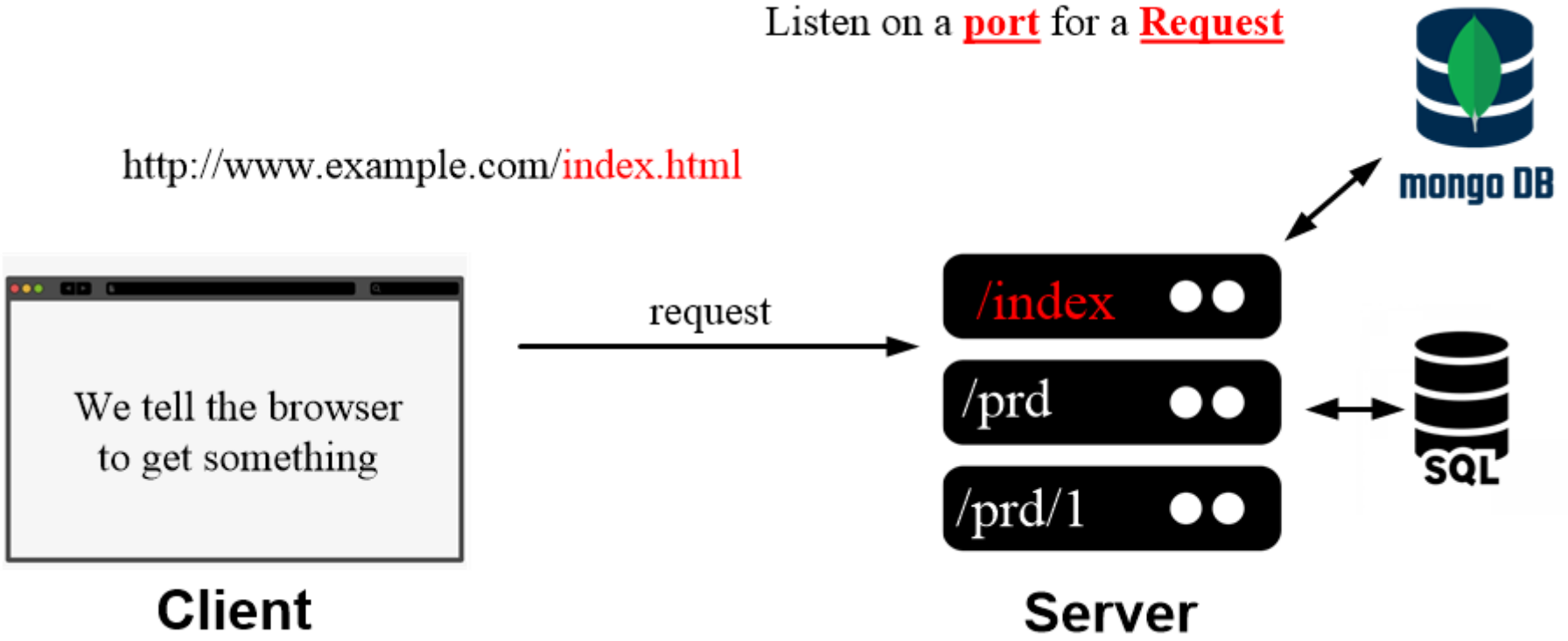


How the back-end works

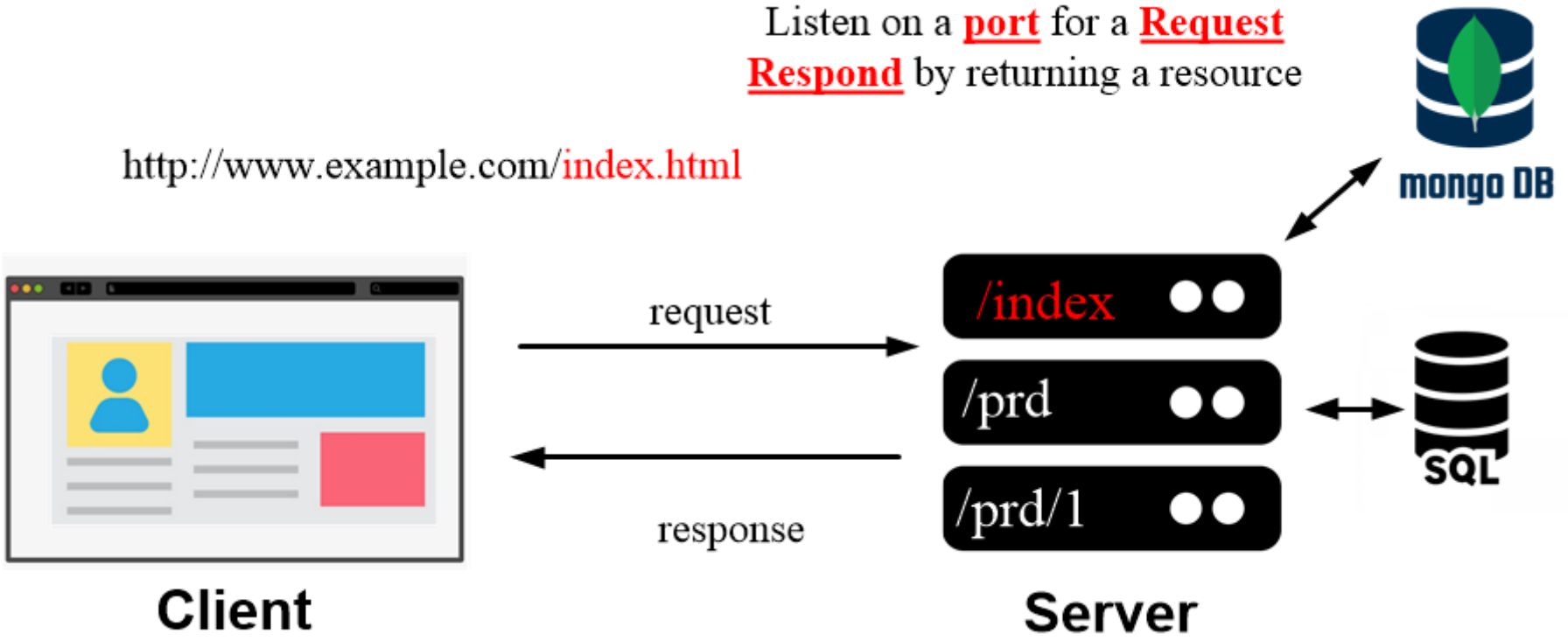
How the back-end works



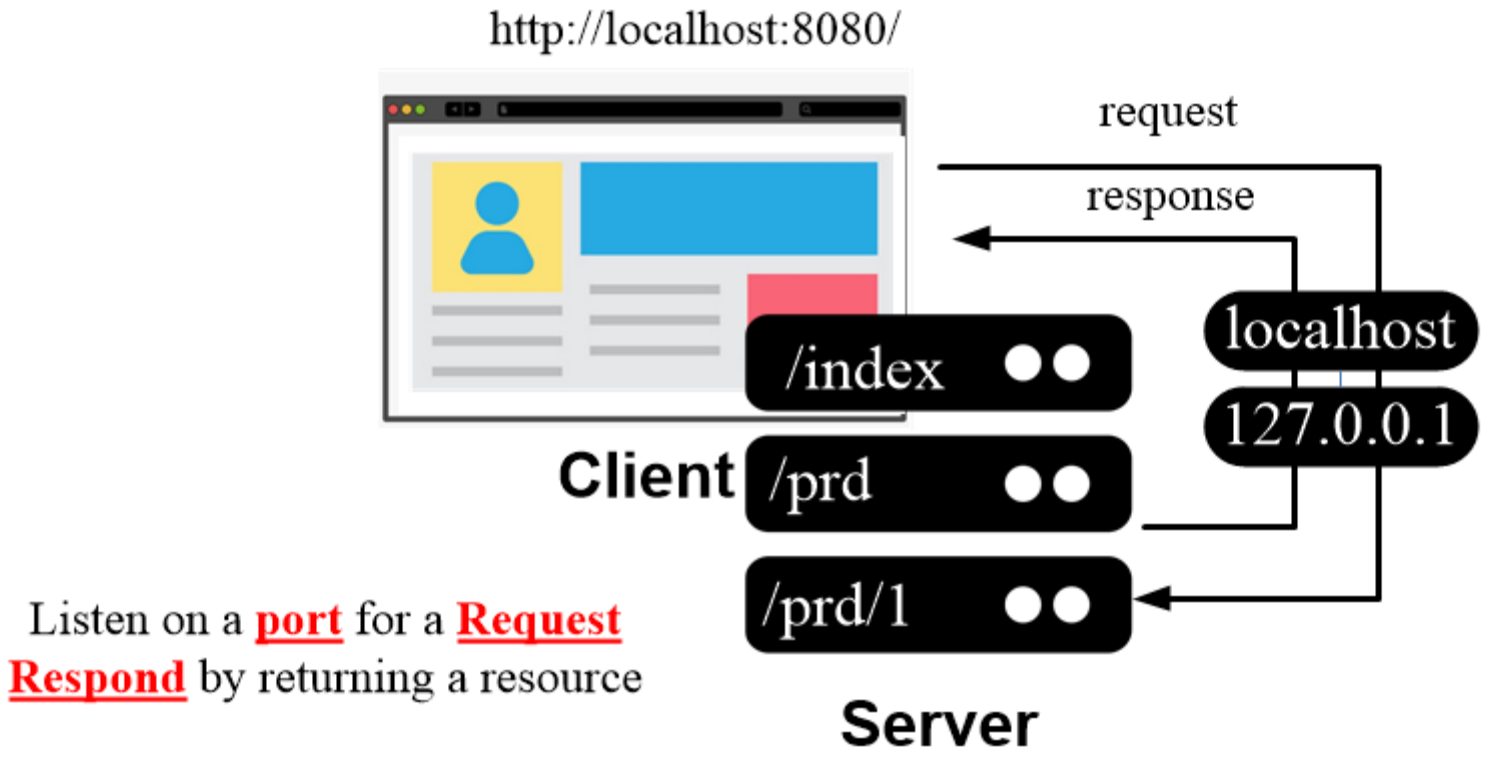
How the back-end works



How the back-end works



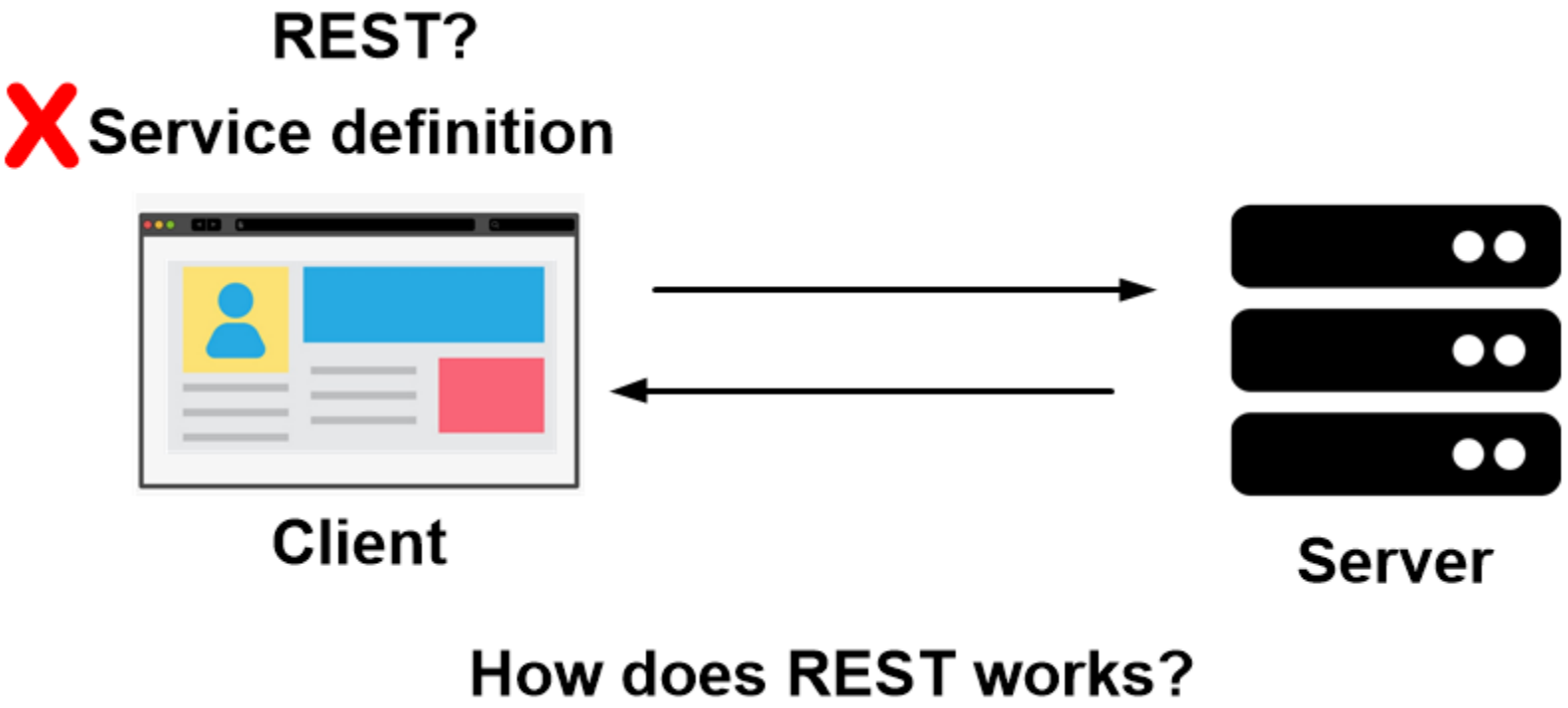
How the back-end works



Localhost (127.0.0.1)

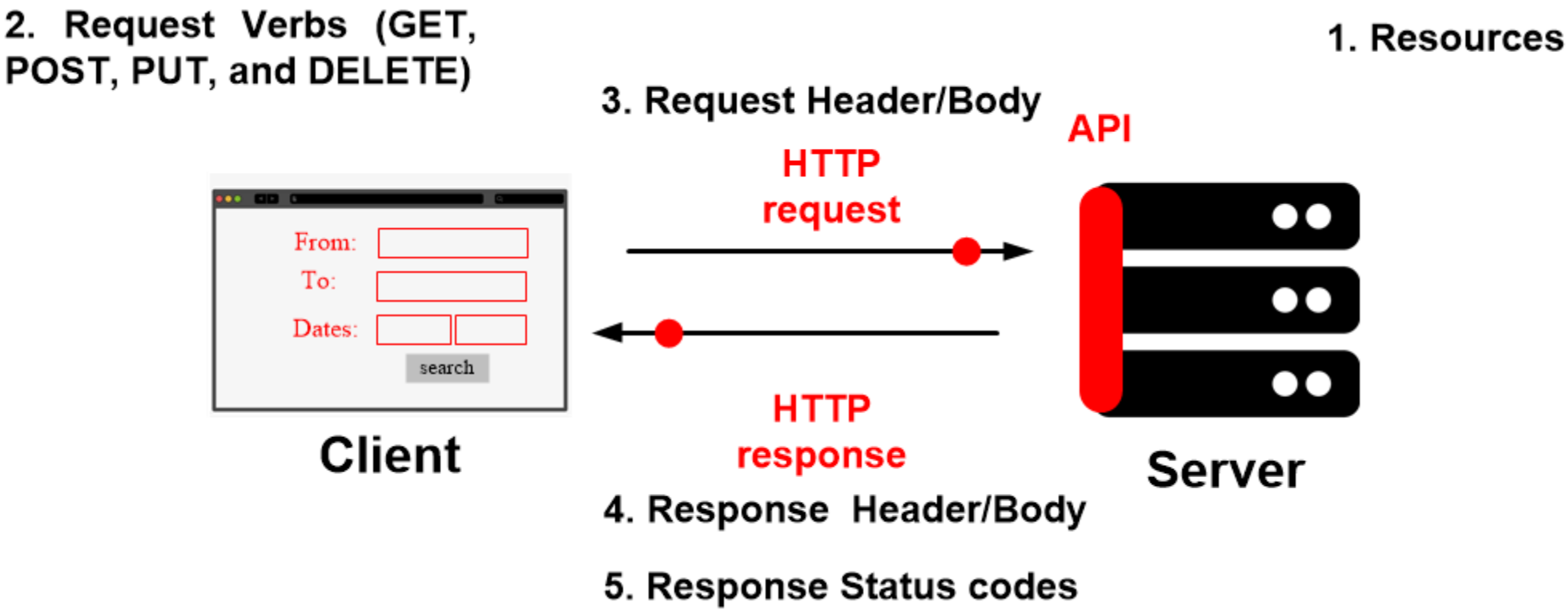
Back to REST

REST - Representational State Transfer



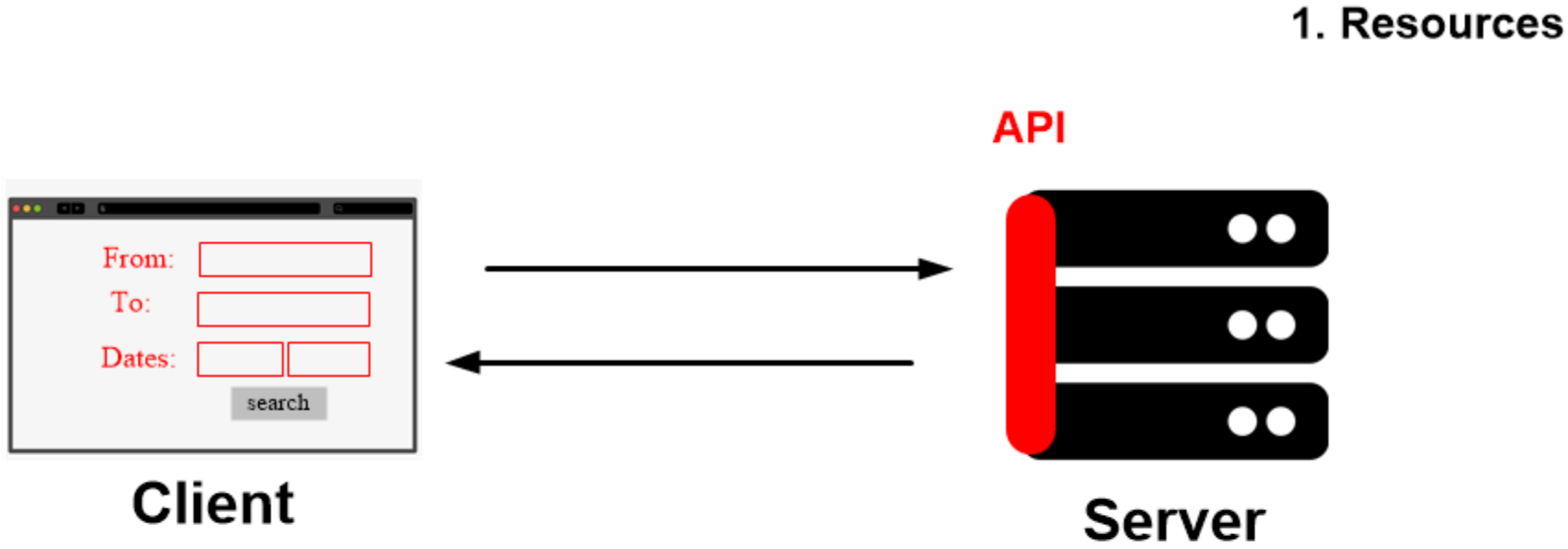
REST is a term coined by ***Roy Fielding*** to describe ***an architecture style*** to be used for "creating" ***Web services***.

REST - Representational State Transfer



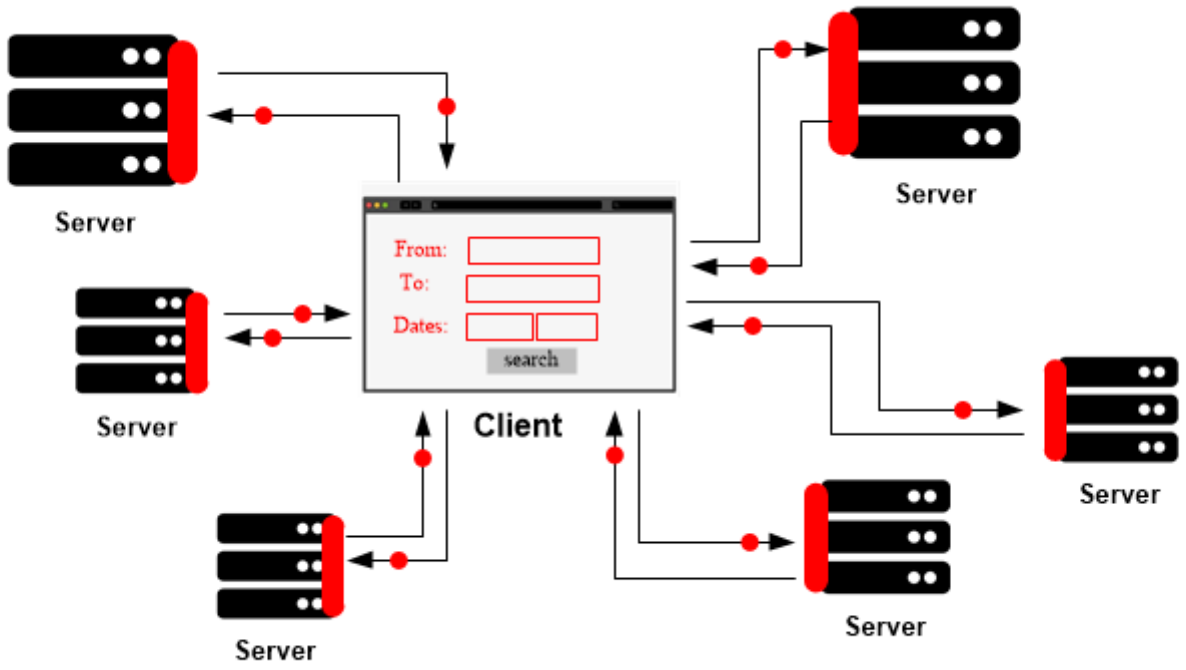
Representational state transfer (REST) is a software architectural style designed for distributed hypermedia, which defines a set of constraints to be used for creating Web services.

1. Resources - API



Application Programming Interface (API) *is a set of definitions and protocols for building and integrating application software.*

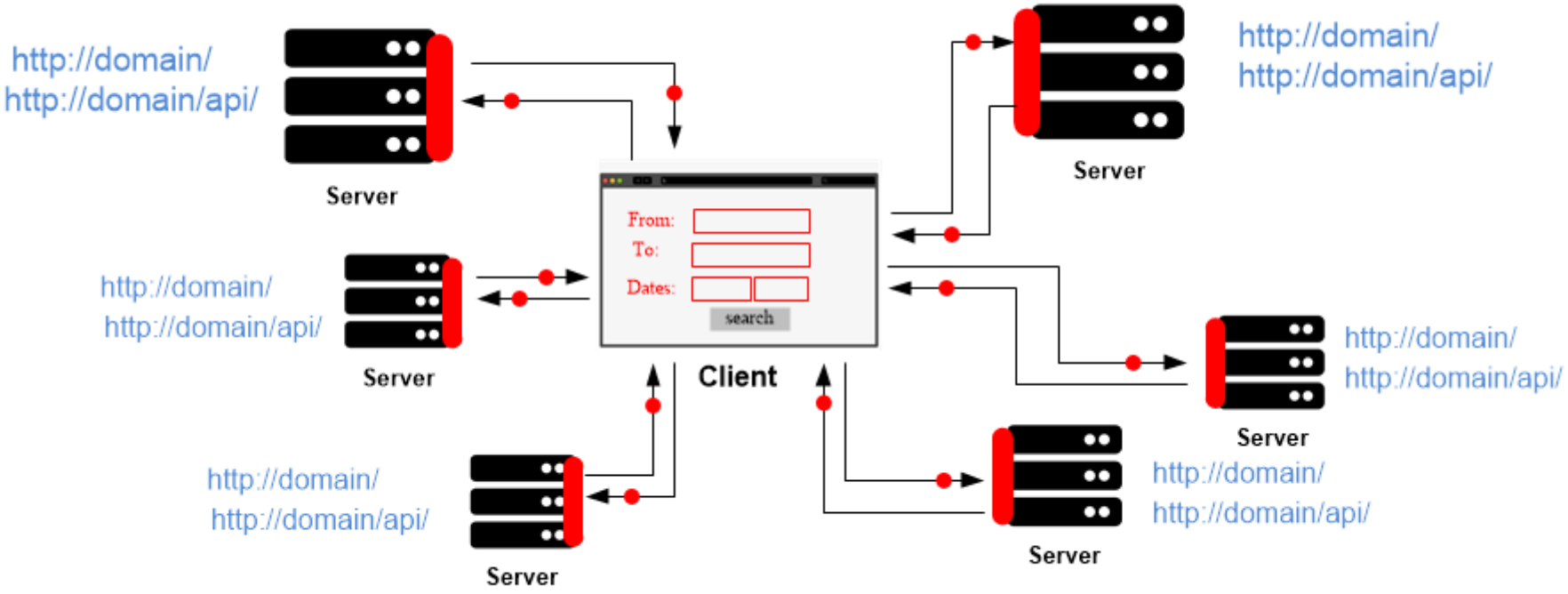
1. Resources - API



API is a software "***interface***", offering a ***service*** to other software(s).

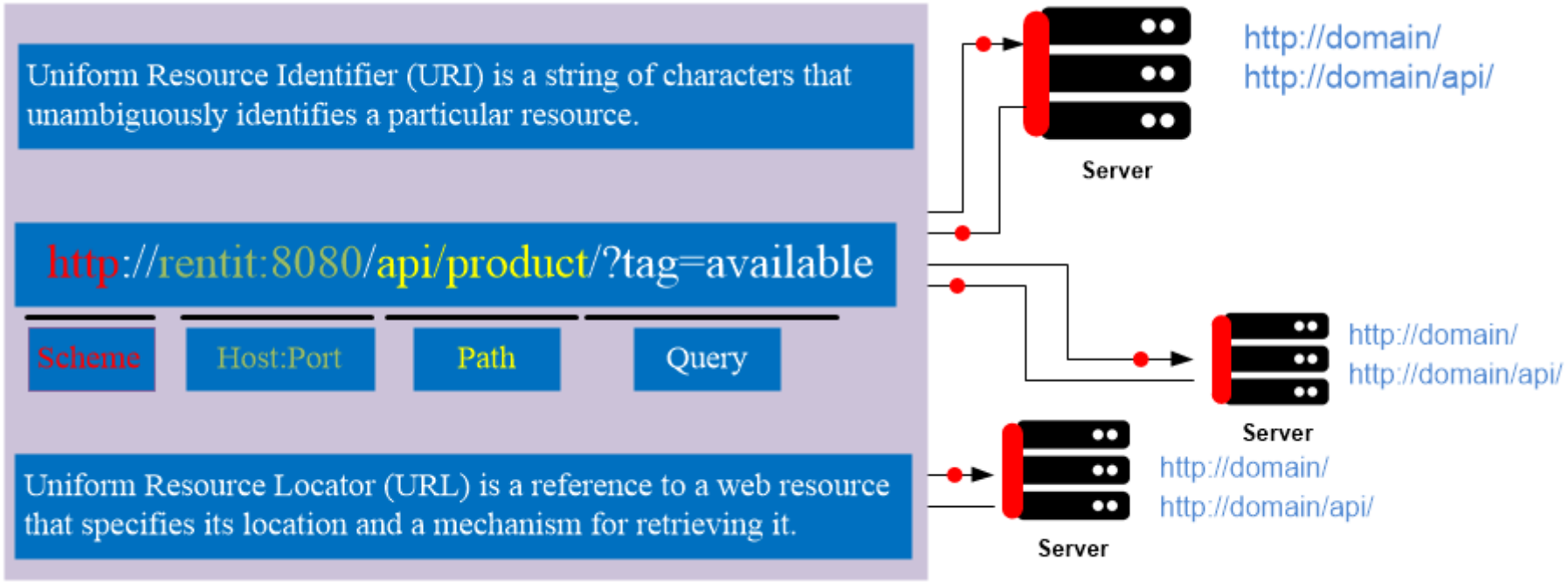
API is a software ***intermediary*** that allows two applications to talk to each other.

1. Resources - API



Uniform Resource Identifier (URI) is a string of characters that unambiguously identifies a particular resource.

1. Resources - API

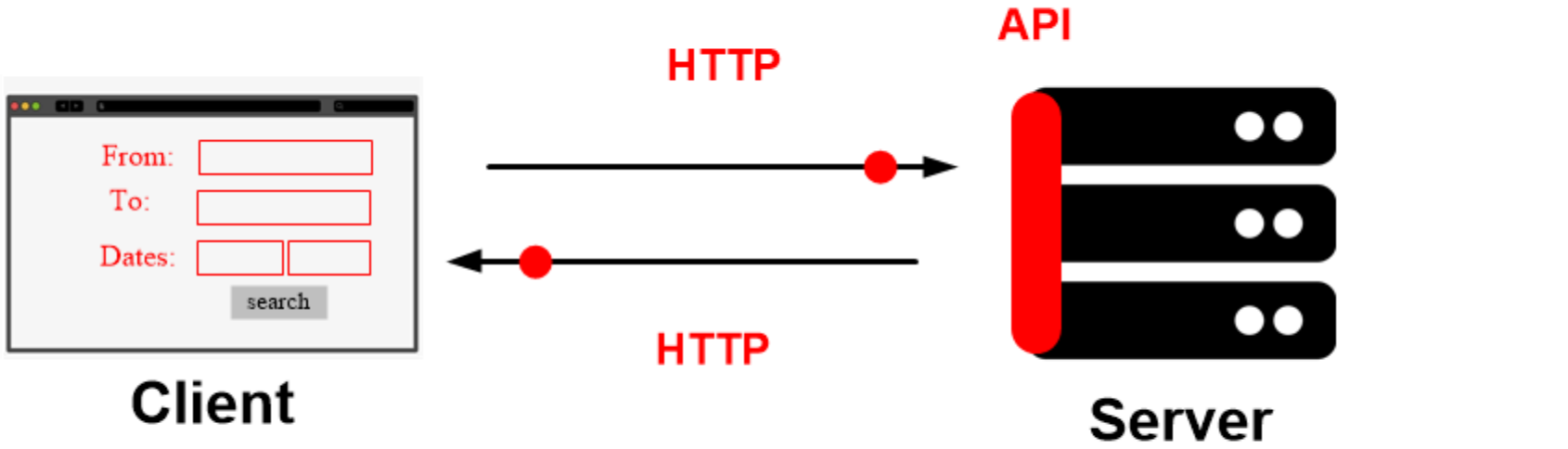


Uniform Resource Identifier (URI) is a string of characters that unambiguously identifies a particular resource.

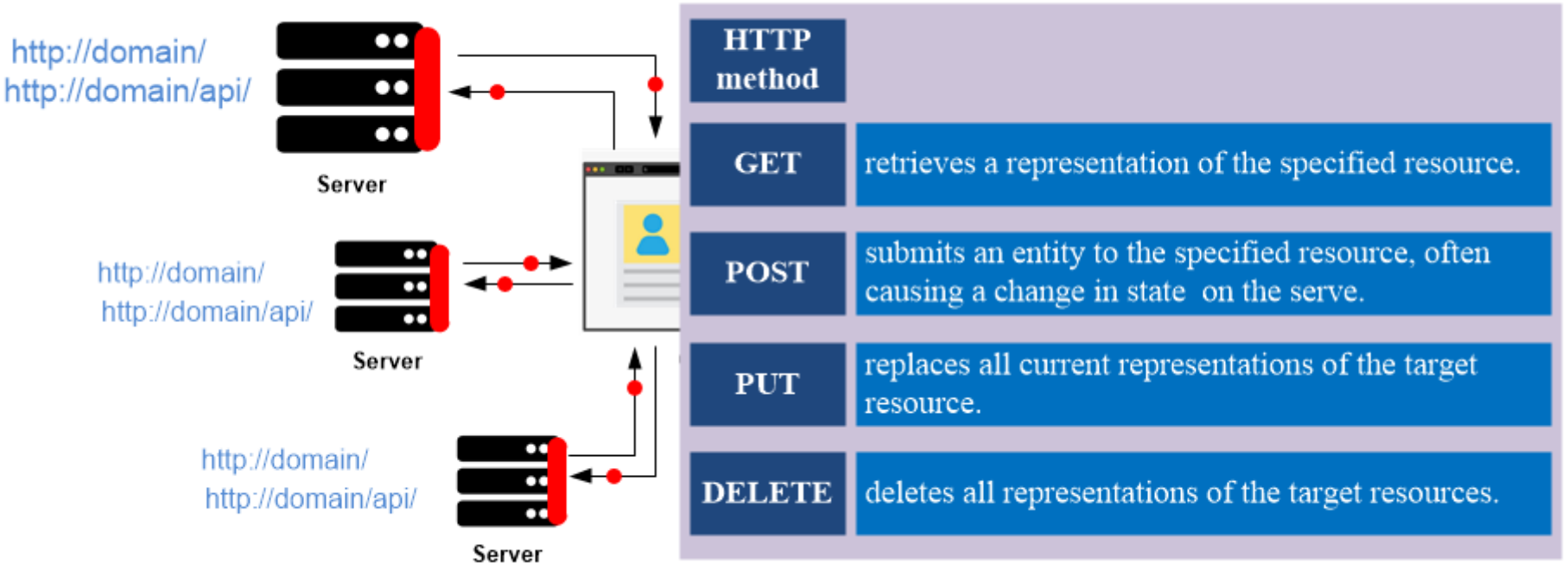
2. Request Verbs

2. Request Verbs (GET, POST, PUT, and DELETE)

1. Resources

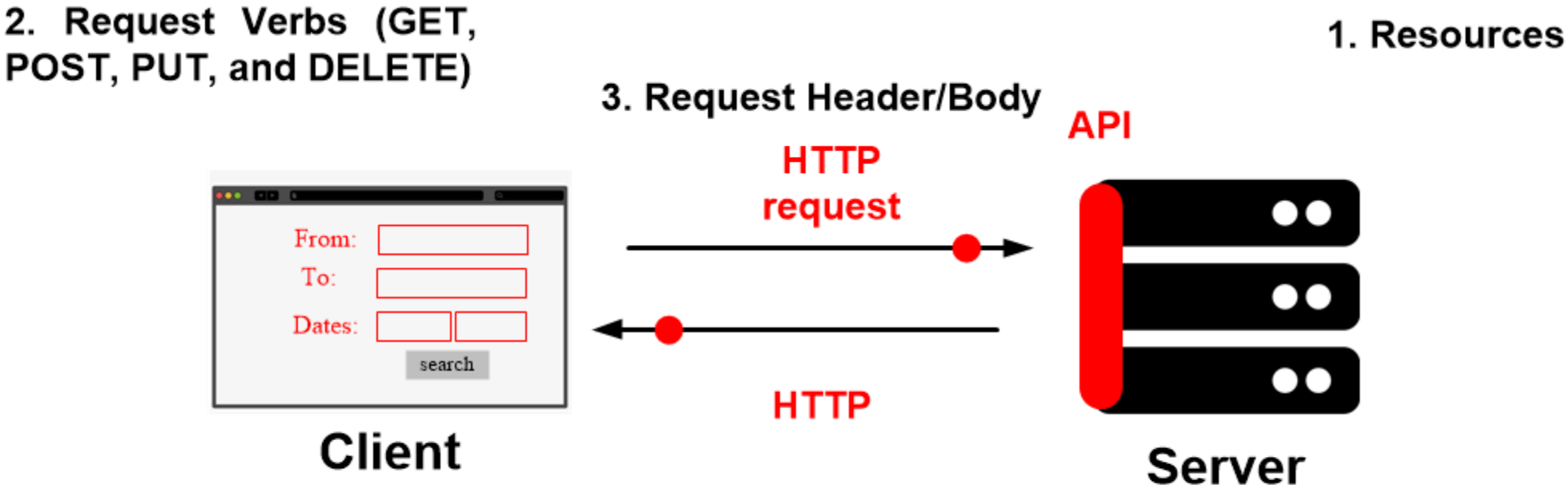


2. Request Verbs



Request verbs (HTTP verbs/ HTTP methods) indicates the desired action to be performed for a given resource.

3. Request Header/Body



3. Request Header/Body

Request header

[Method][Path][Protocol]
Host:
Content-Type:
Accept:
Date:

Get / HTTP/1.1
Host: www.ut.ee
Content-Type: application/json
Accept: application/json
Date: Mon, 09 May 2022 ..

Request body

A request header can be used to provide information (meta-data) about the request context.

3. Request Header/Body

Request header

[Method][Path][Protocol]
Host:
Content-Type:
Accept:
Date:

POST /orders HTTP/1.1
Host: www.ut.ee
Content-Type: application/json
Accept: application/json
Date: Mon, 09 May 2022 ..

Request body

```
{
 "order_Id": 01,
 "quantity": 10
}
```

A request body contains the actual message to be delivered.
Not all requests have body (e.g., **GET, DELETE).**

4. Response Header/Body

Request header

[Method][Path][Protocol]
Host:
Content-Type:
Accept:
Date:

Get / HTTP/1.1
Host: www.ut.ee
Content-Type: application/json
Accept: application/json
Date: Mon, 09 May 2022 ..

Response header

[Protocol][Code][Reason]
Date:
Server:
Content-Type:
Last modified:

HTTP/1.1 200 OK
Date: Mon, 21 May 2022 ..
Server: Apache/2.2.14 (Win32)
Content-Type: application/json
Last modified: Tue, 04 May ..

A response header can be used to provide information (meta-data) about the request context.

4. Response Header/Body

Response header

[Protocol][Code][Reason]
Date:
Server:
Content-Type:
Last modified:

HTTP/1.1 200 OK
Date: Mon, 21 May 2022 ..
Server: Apache/2.2.14 (Win32)
Content-Type: application/json
Last modified: Tue, 04 May ..

Response body

```
{
 "order_Id": 01,
 "quantity": 10,
 "product_Id": 03,
}
```

A response body contains the actual message to be delivered.
Not all responses have body (e.g., **PUT, DELETE).**

5. Response Status Codes

Response codes classes:

1XX - Informational
2XX - Successful
3XX - Redirection
4XX - Client Error
5XX - Server Error

Common response codes:

200 – OK
301 - Moved to new URL
304 – Not modified (Cached version)
400 - Bad Request
401 - Unauthorized
403 - Forbidden
404 - Not found
500 - Internal Server Error
502 - Bad Gateway
503 - Service Unavailable

HTTP response status codes indicate the status of an HTTP.

Final remarks

REST is NOT a

A Framework

A Technology

Standard specifications

A Protocol

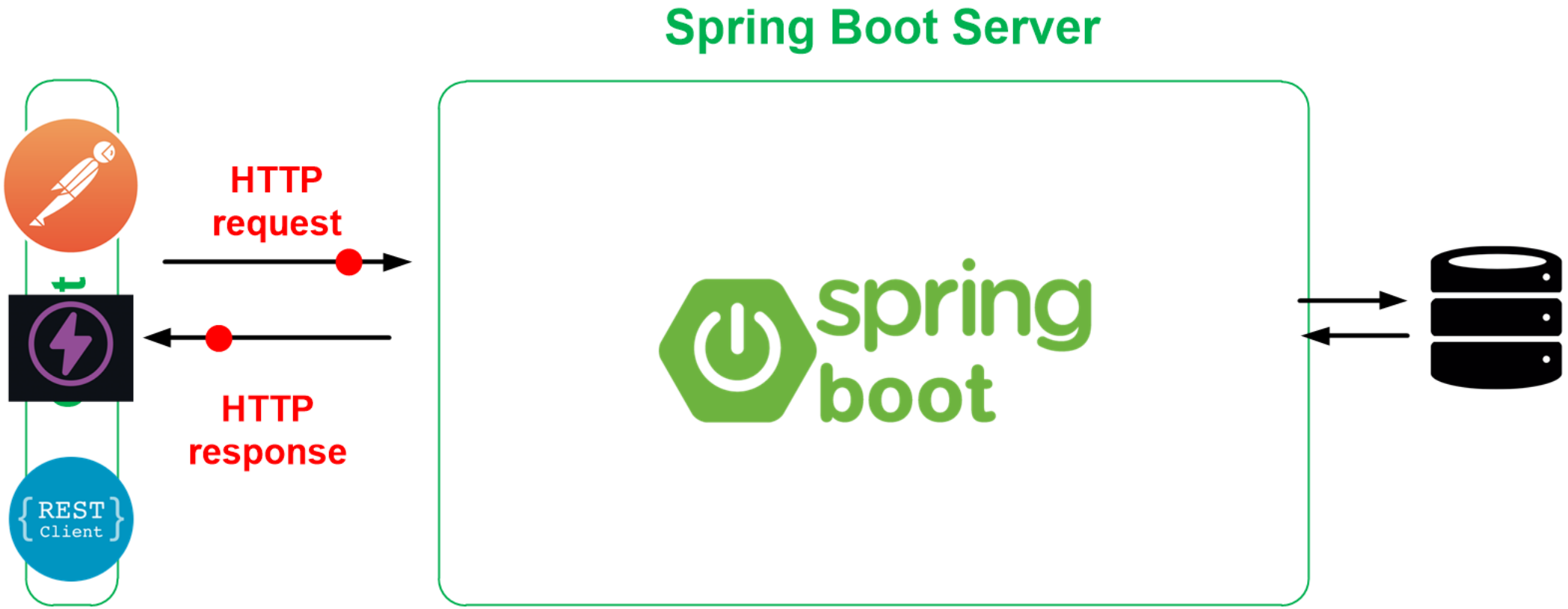
REST is

- ***REST*** is a set of guidelines and architectural principles, which talks about how should a client interact with a server.
- ***RESTful*** is the implementation of ***REST*** guidelines
- ***RESTful*** is Web Services

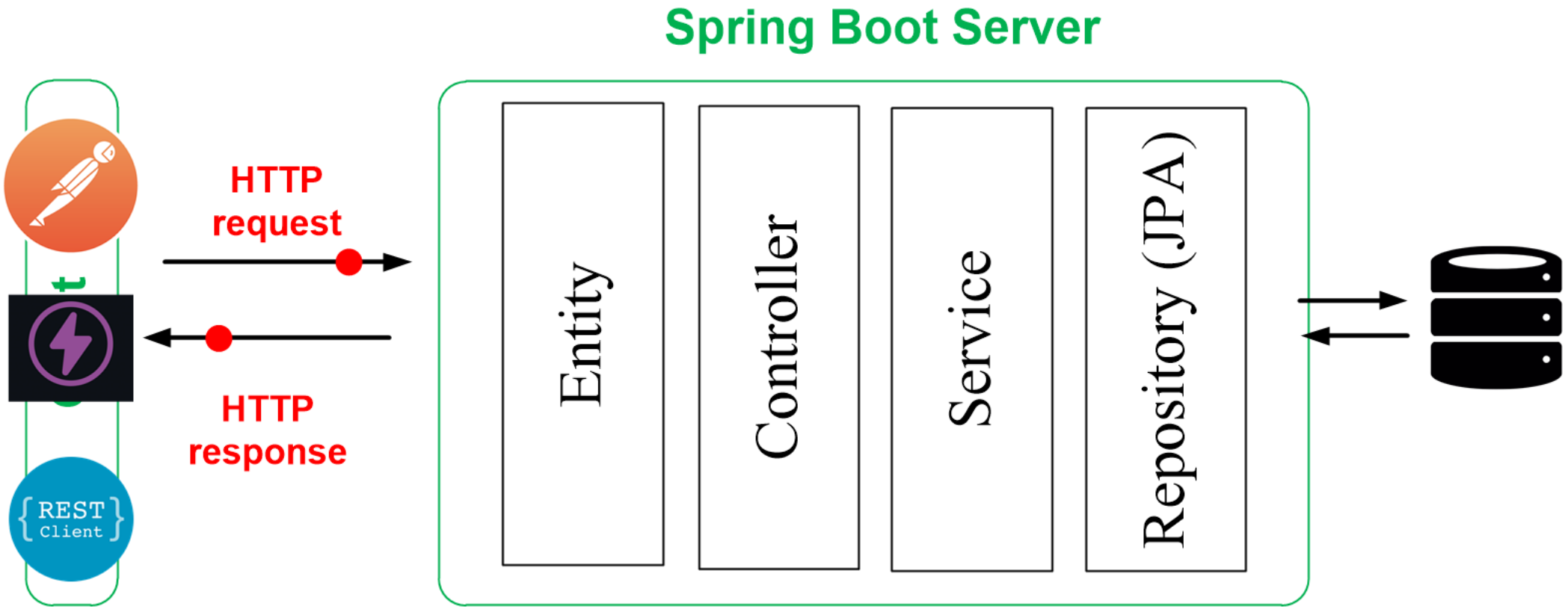
REST – the full picture

REST – the full picture

REST – the full picture



REST – the full picture



REST – practical example

REST – practical example

In this example, we have two entities:

- Product
- Order

REST – practical example

In this example, we have two entities:

- Product
- Order

esi.~.products

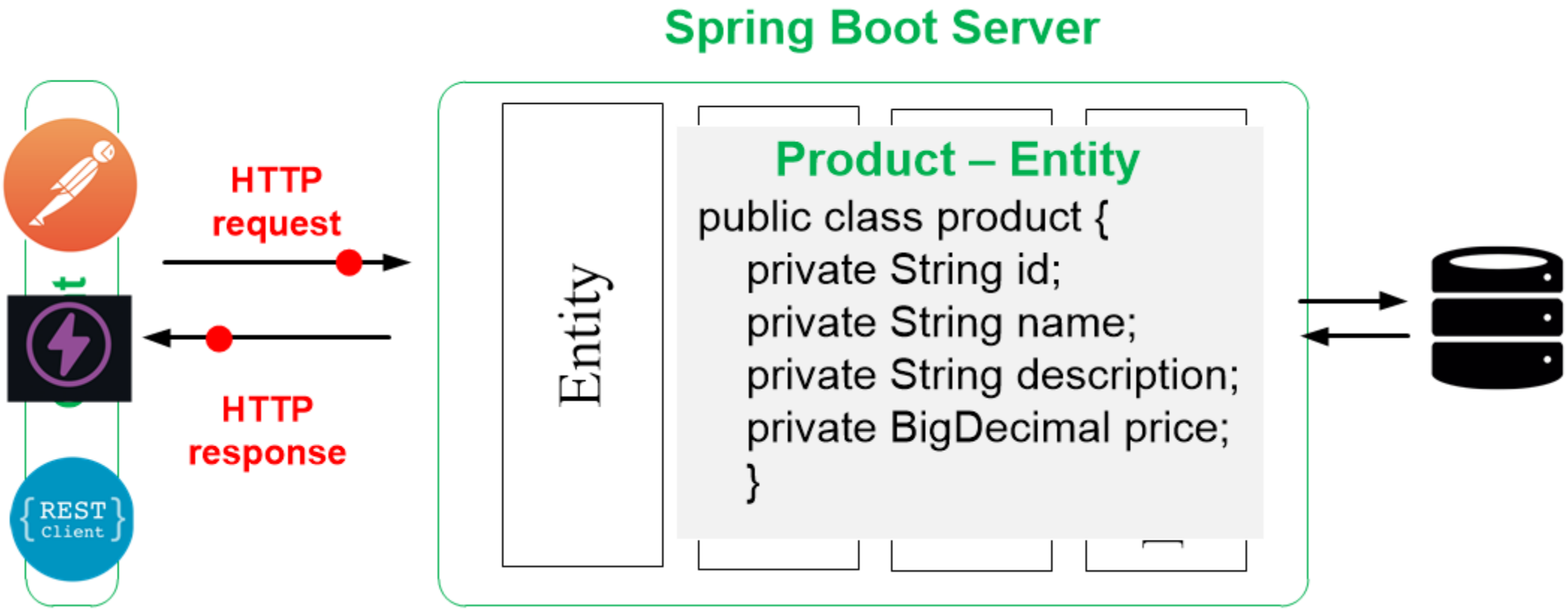
Product (Entity)

ProductController

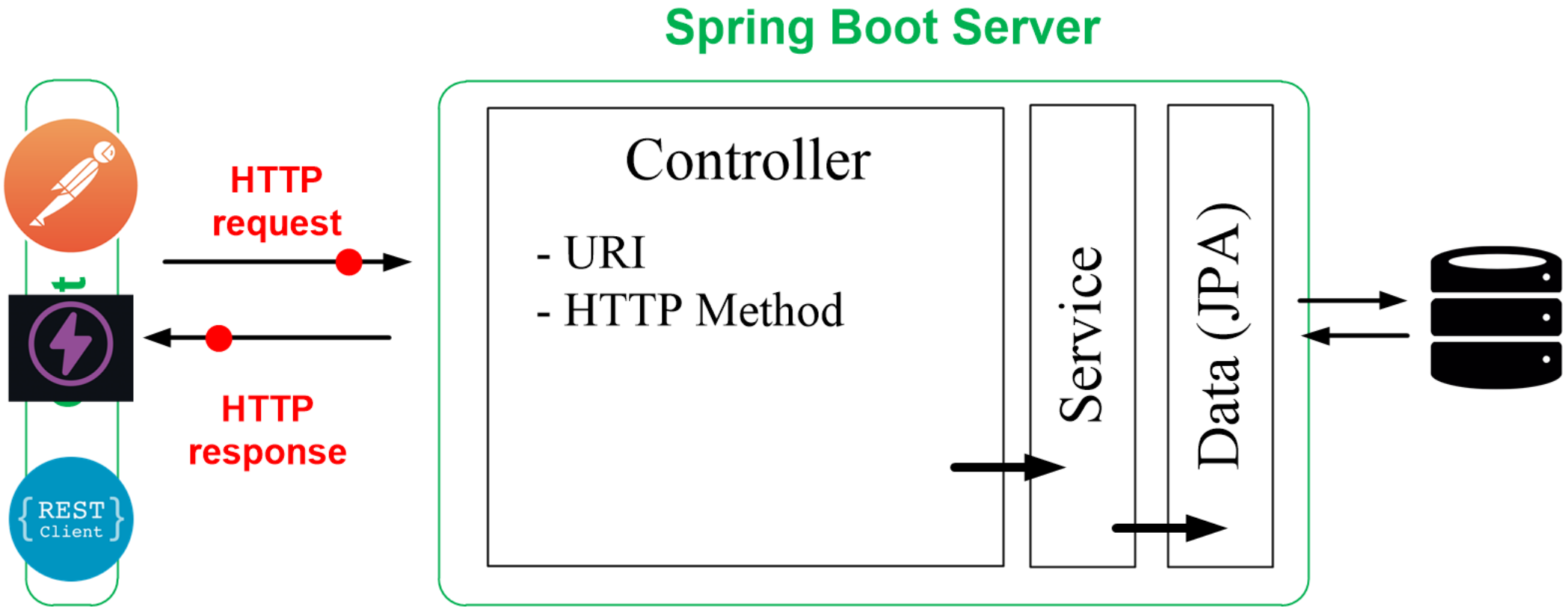
ProductService

ProductRepository

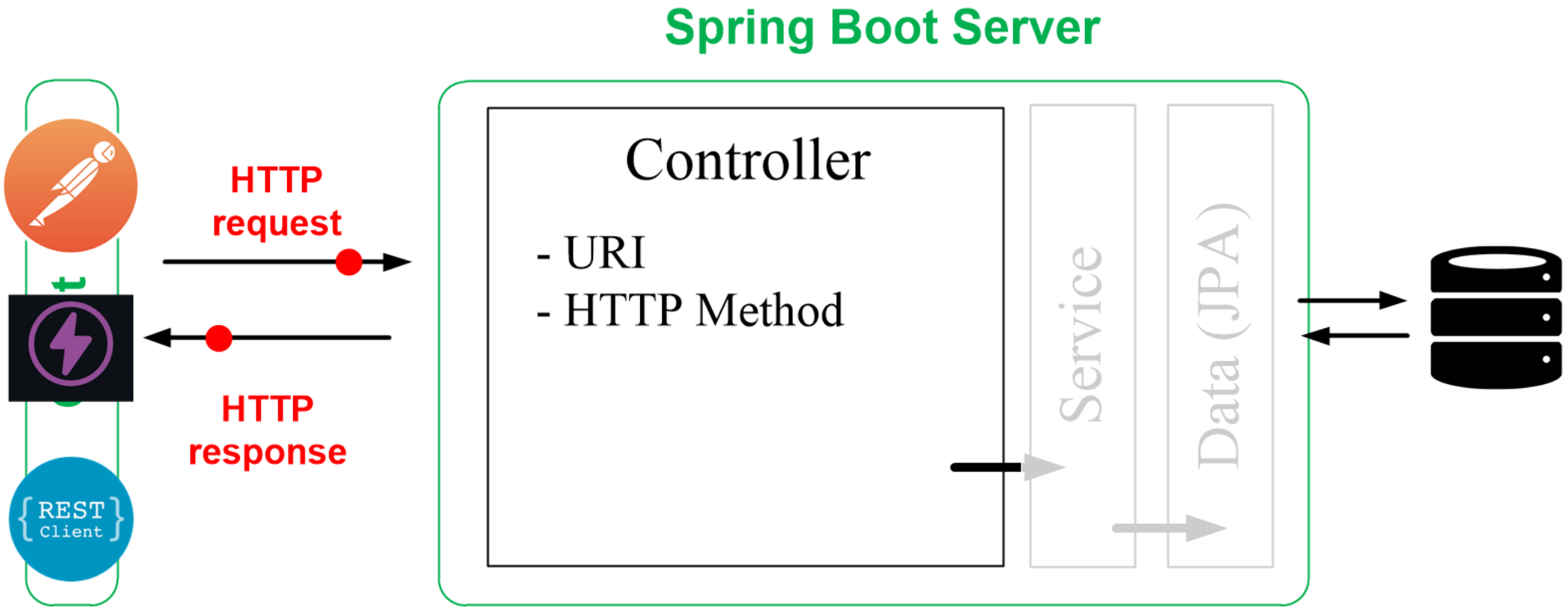
REST – practical example



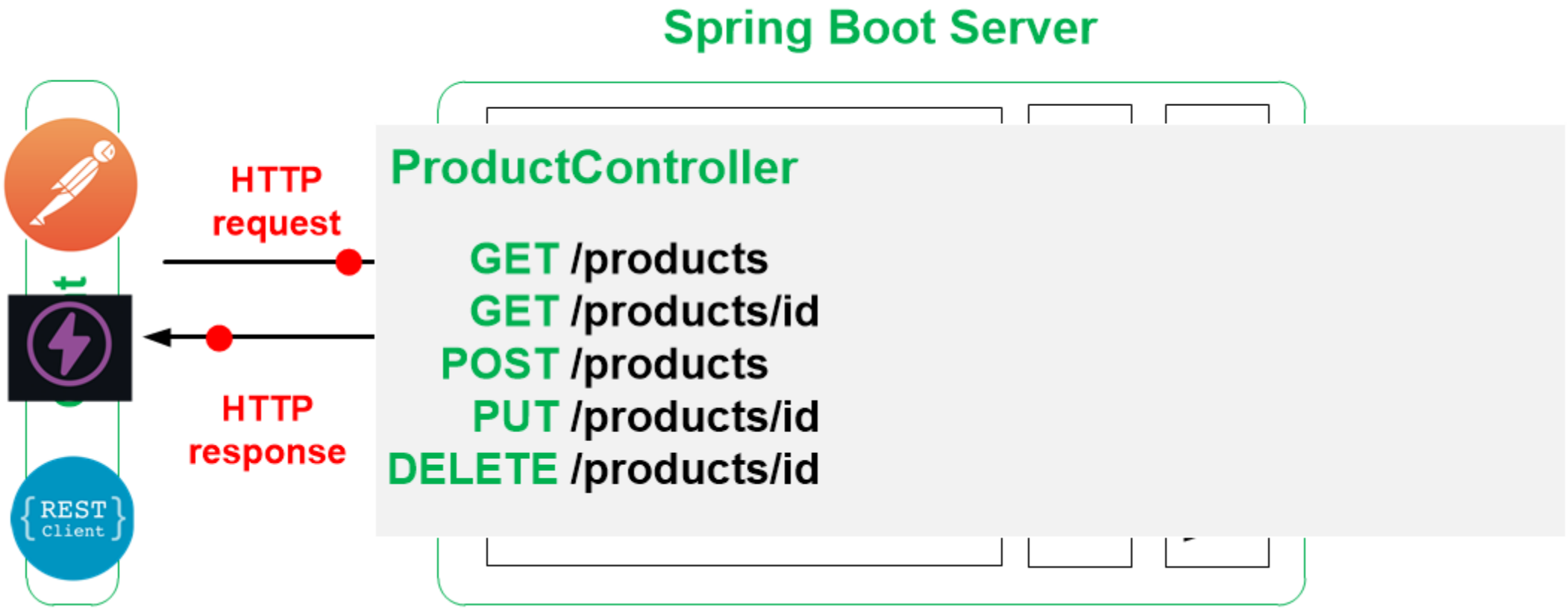
REST – practical example



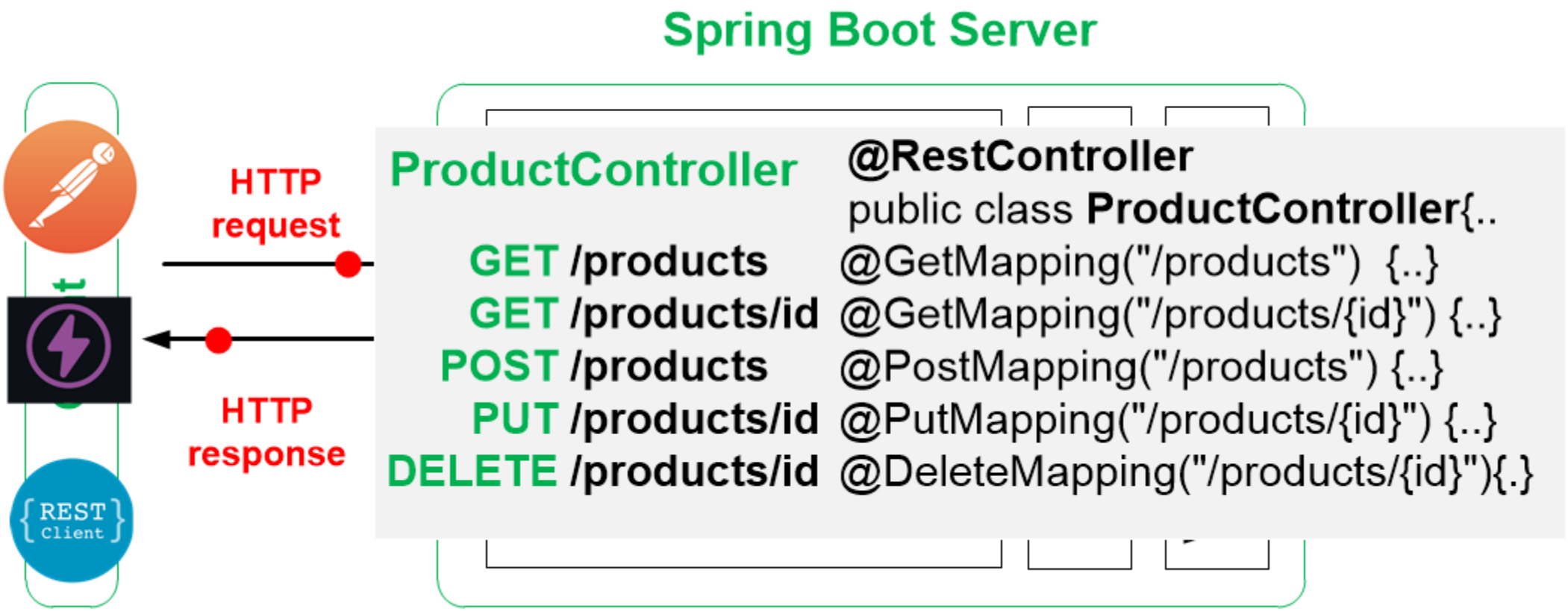
REST – practical example



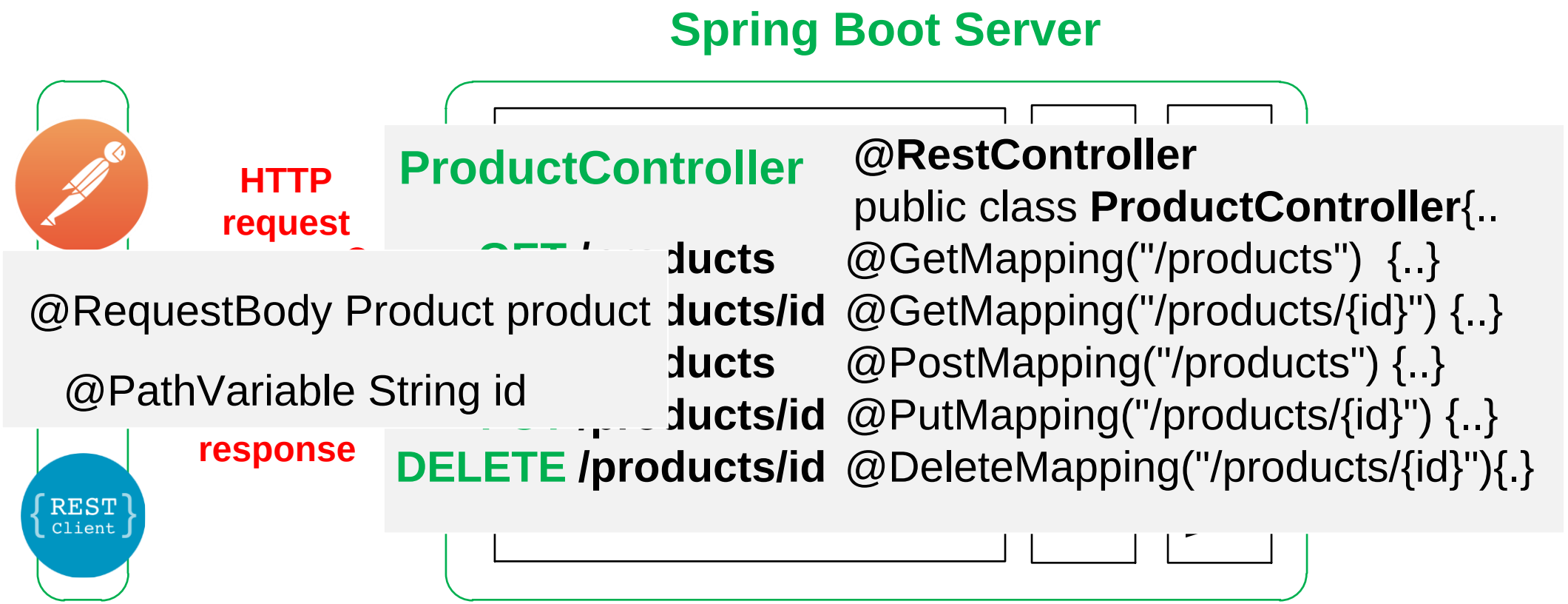
REST – practical example



REST – practical example



REST – practical example



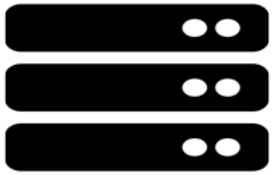
Note: Session 2.1: Spring Boot - Rest API - [Pre Lab] and **Session 2.2:** Postman - [Pre Lab], in the Course Wiki provide practical examples covering Rest API

RESTful API – CRUD Operations - I

Model-View-Controller (MVC)

What is Model-View-Controller

Client

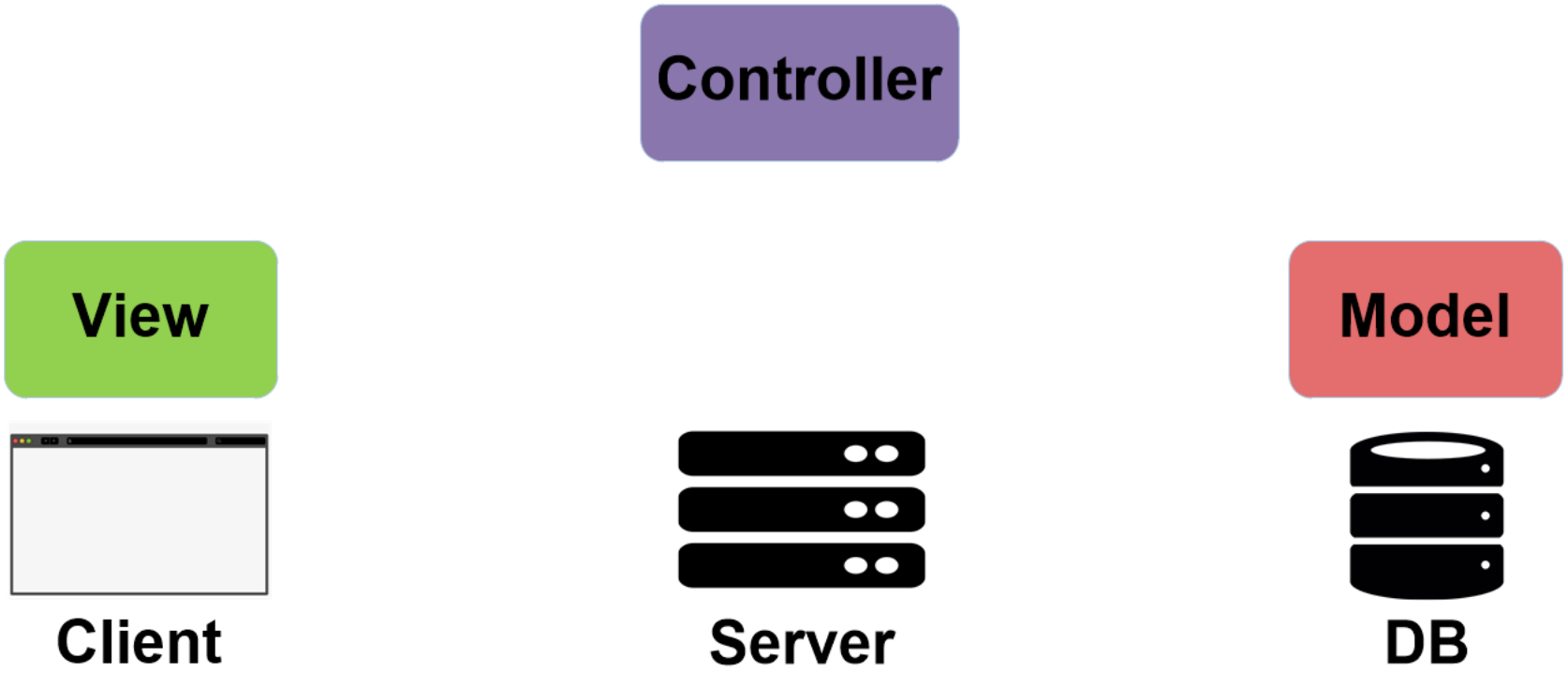


Server

DB

MVC is a **design pattern** for software projects, where an application is divided into **three** interconnected parts:

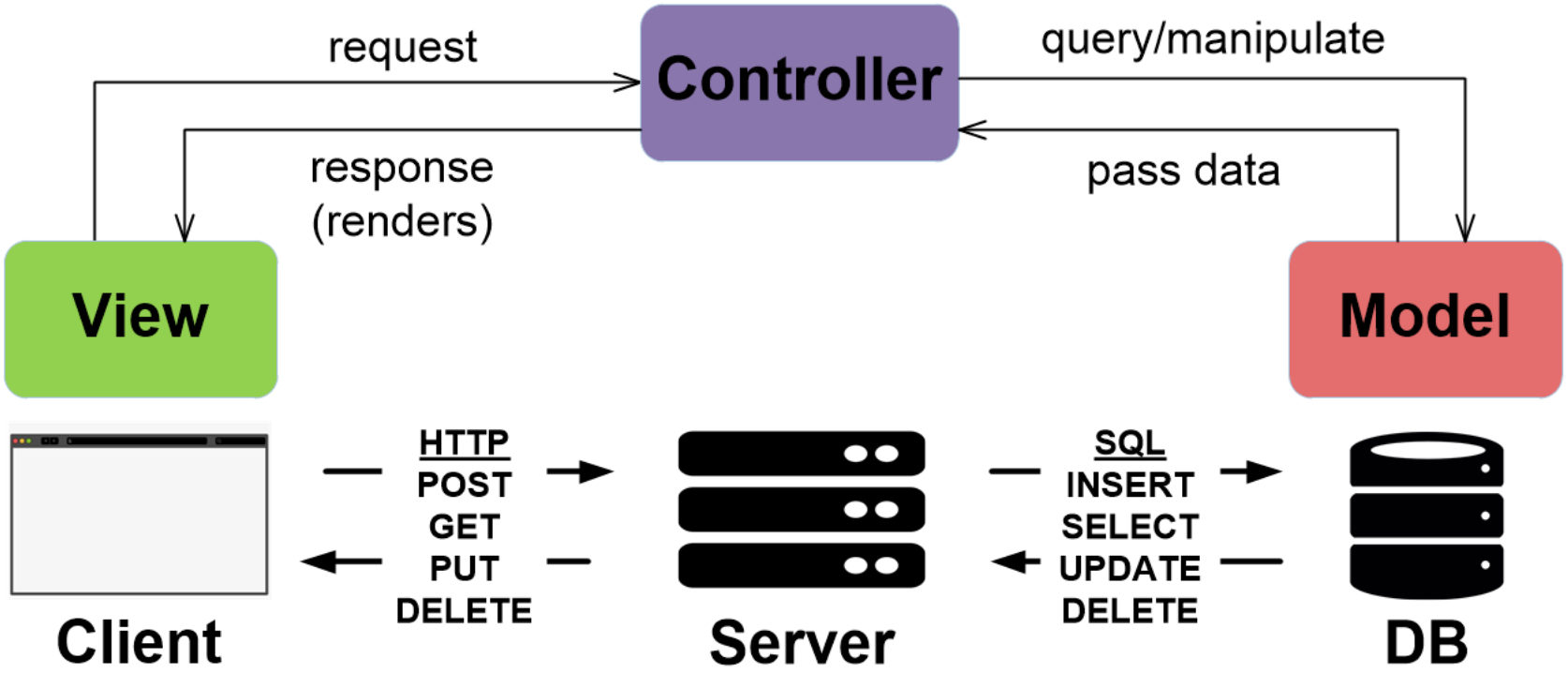
What is Model-View-Controller



MVC is a **design pattern** for software projects, where an application is divided into **three** interconnected parts:

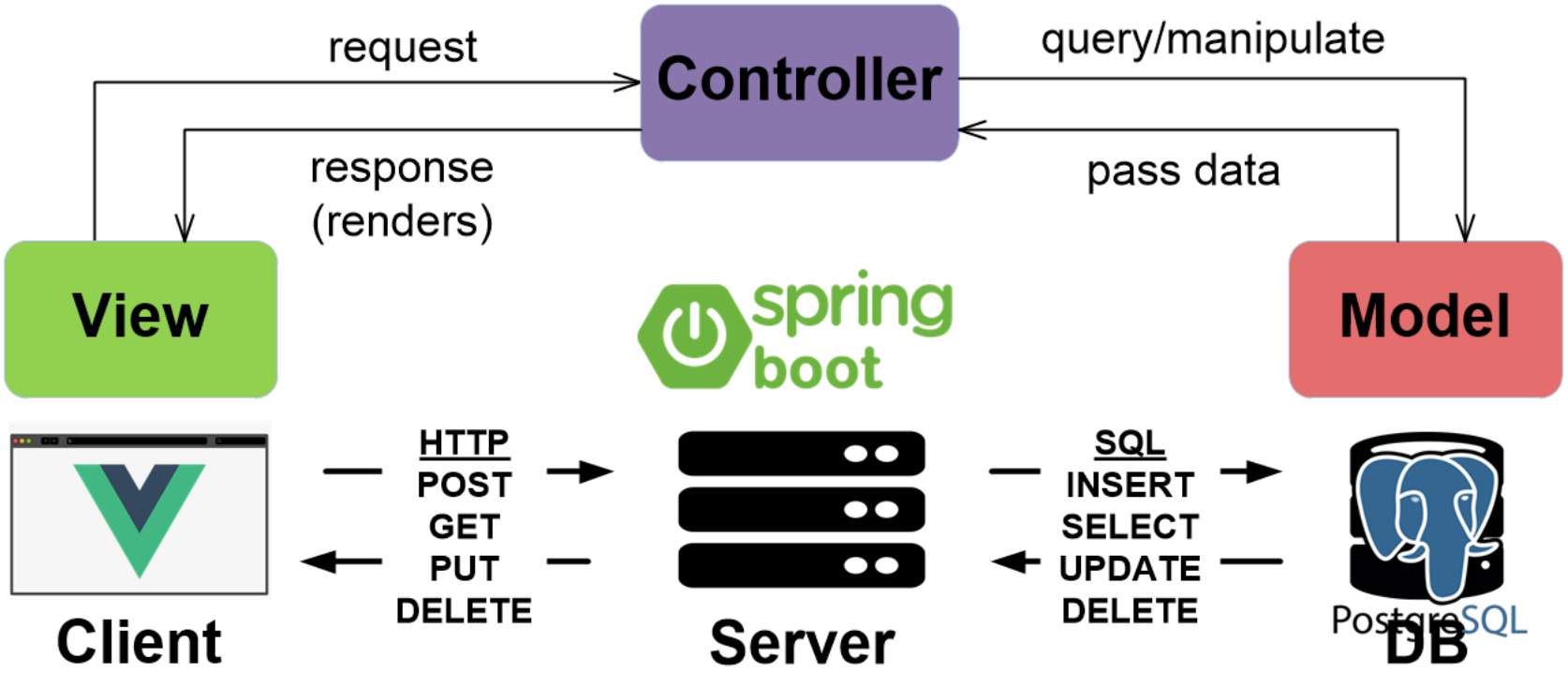
- **Model** is the data part.
- **View** is the user Interface part.
- **Controller** is the request-response handler part.

What is Model-View-Controller



In **MVC** web applications, the user will typically **request** a **resource** from the **server**, (**Controller**), which may cause the **controller** to **request** application **data** from the database (**Model**). Then, pass the **data/resource** to the Client (**View**), which will finally format the data for the end user.

Building an MVC application

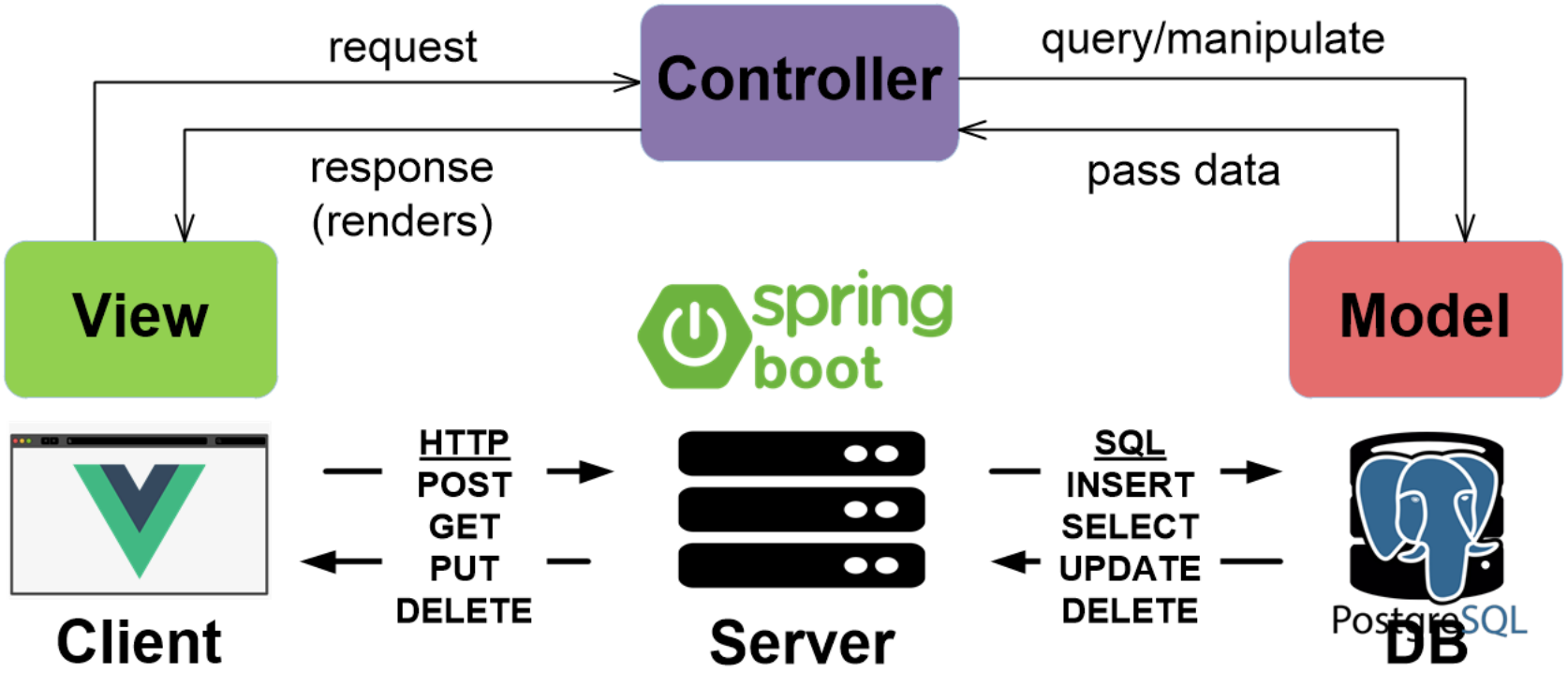


The View - a **front-end** (**Vue.js**) that provides the user interface.

The Controller - a **backend** (**Spring Boot**) allows for handling front-end requests.

The Model - a **database** (Postgres) that contain the data of our App.

Building an MVC application



The View - a **front-end** (**Vue.js**) that provides the user interface.

The Controller - a **backend** (**Spring Boot**) allows for handling front-end requests.

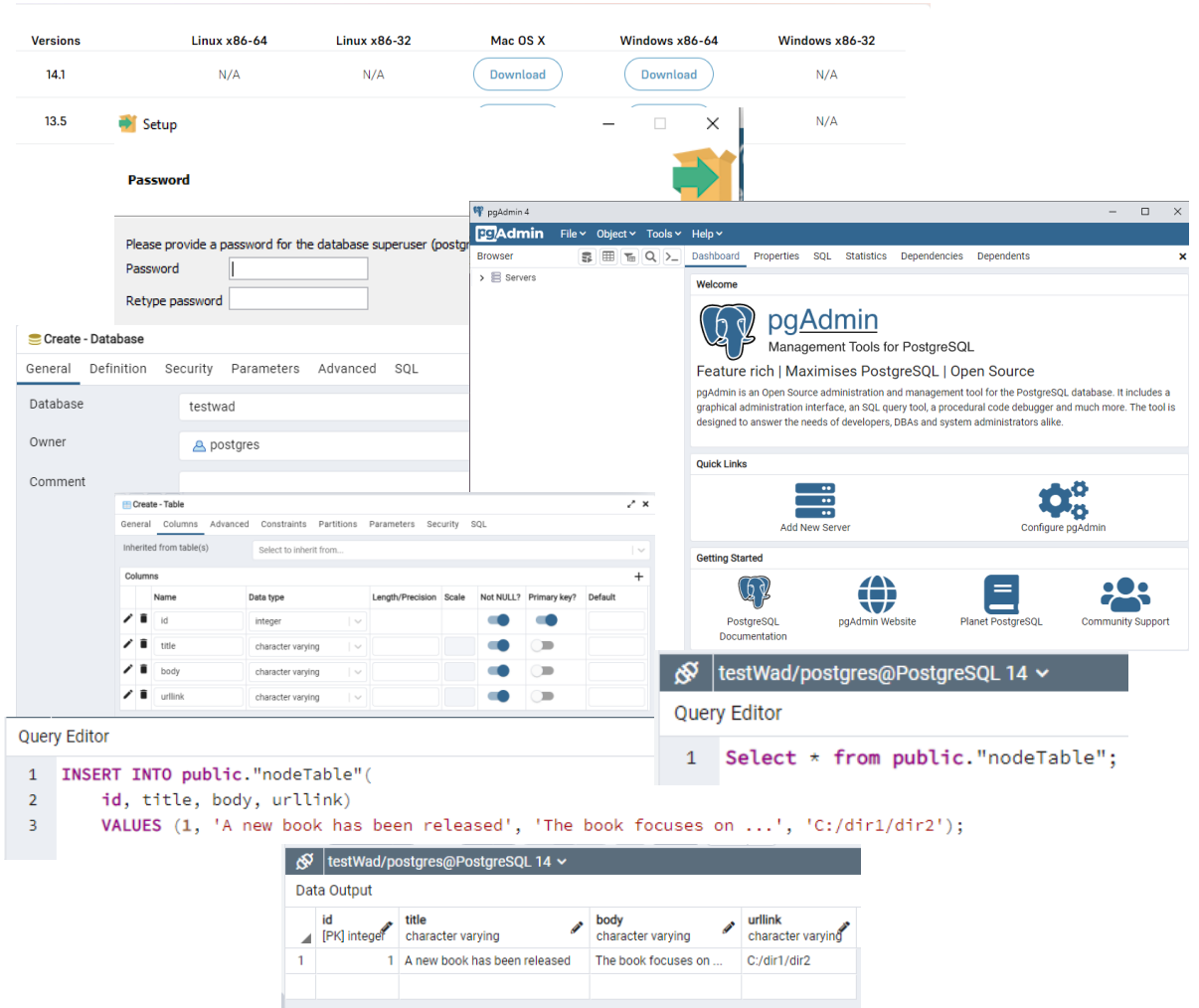
The Model - a **database** (Postgres) that contain the data of our App.

The Model (Databases)

Setting up our database and connecting it to our server

Installing PostgreSQL

Within the material of the **practical session**, you can find **a detailed information** on how to **install** Postgres, creating a database.



Connect Spring Boot to Postgres databse

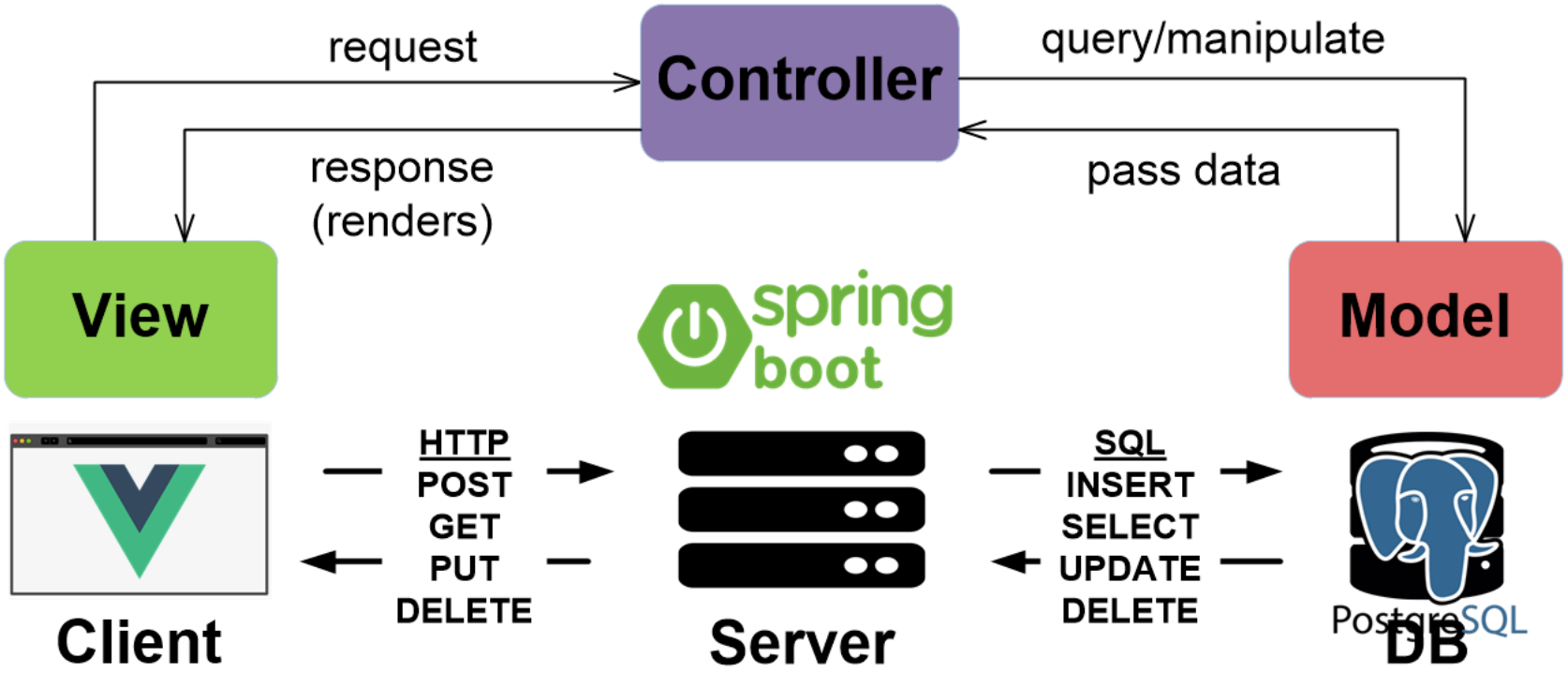
You just need to add the following snippet to /resources/application.properties

```
# /resources/application.properties

# Database Settings
spring.datasource.url=jdbc:postgresql://localhost:5432/esi2023 # database name
spring.datasource.username = postgres # user name
spring.datasource.password = postgres # password

# The SQL dialect makes Hibernate generate better SQL for the chosen database
spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.PostgreSQLDialect

# Hibernate ddl auto (create, create-drop, validate, update)
# Allows for auto creation of tables
spring.jpa.hibernate.ddl-auto = update
```

The Controller

Controller-Service-Repository pattern

Building an MVC application



The View - a **front-end** (**Vue.js**) that provides the user interface.

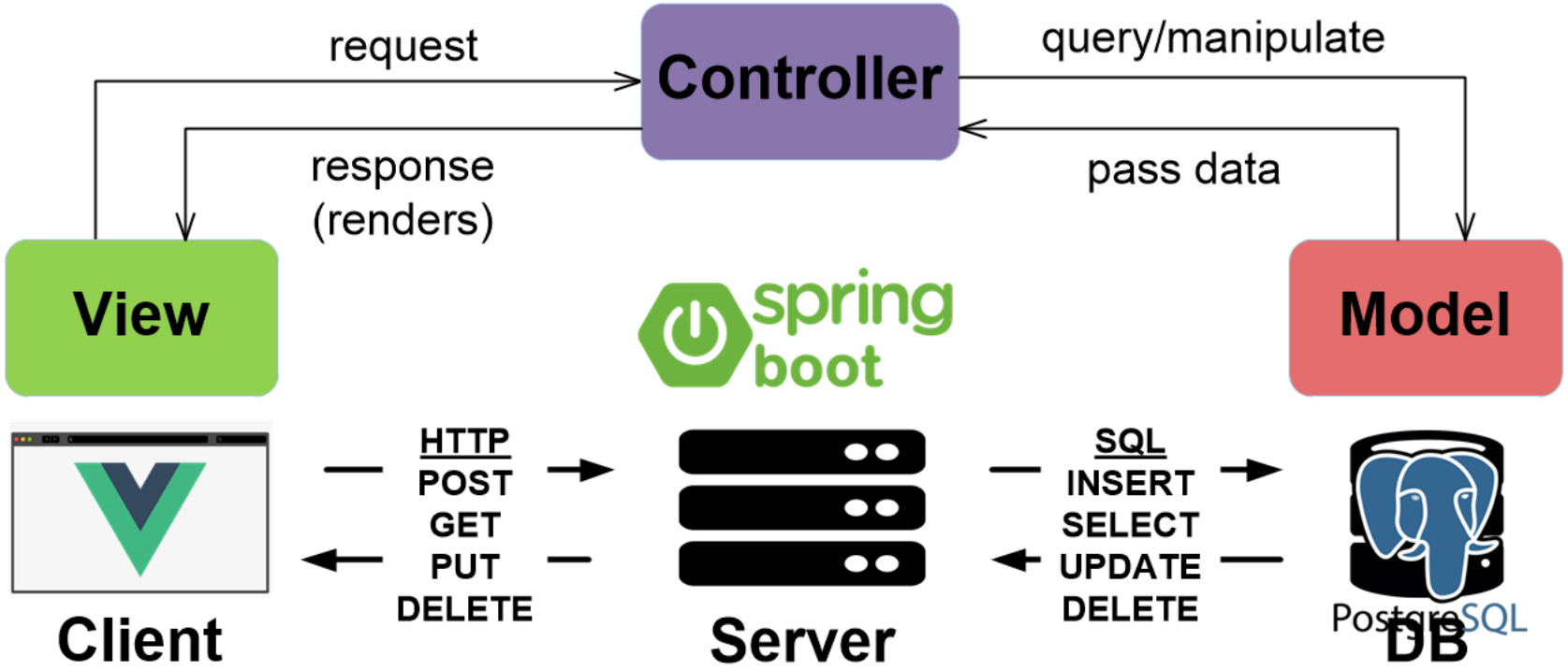
The Controller - a **backend** (**Spring Boot**) allows for handling front-end requests.

The Model - a **database** (Postgres) that contain the data of our App.

CRUD operations

CRUD operations

CRUD
Create
Read
Update
Delete



CRUD operations are the **four basic** operations of persistent storage.

CRUD operations

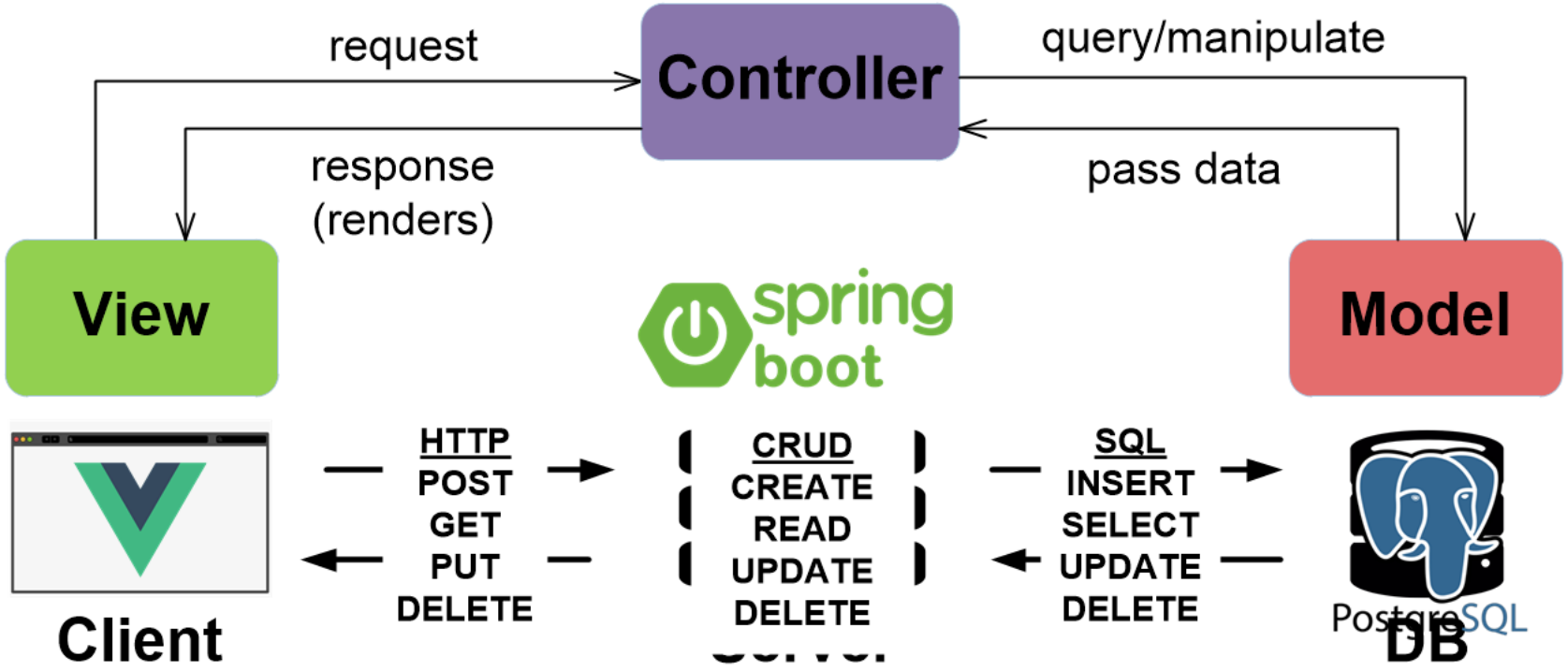
CRUD

Create

Read

Update

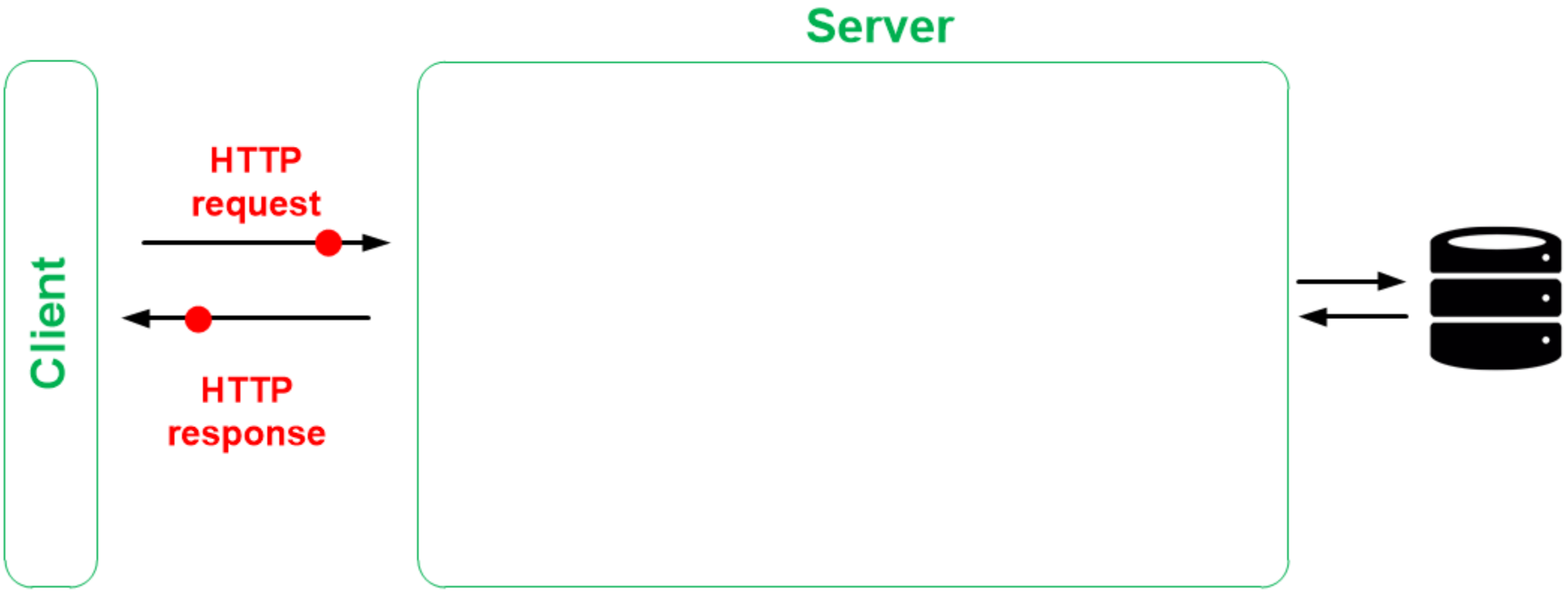
Delete



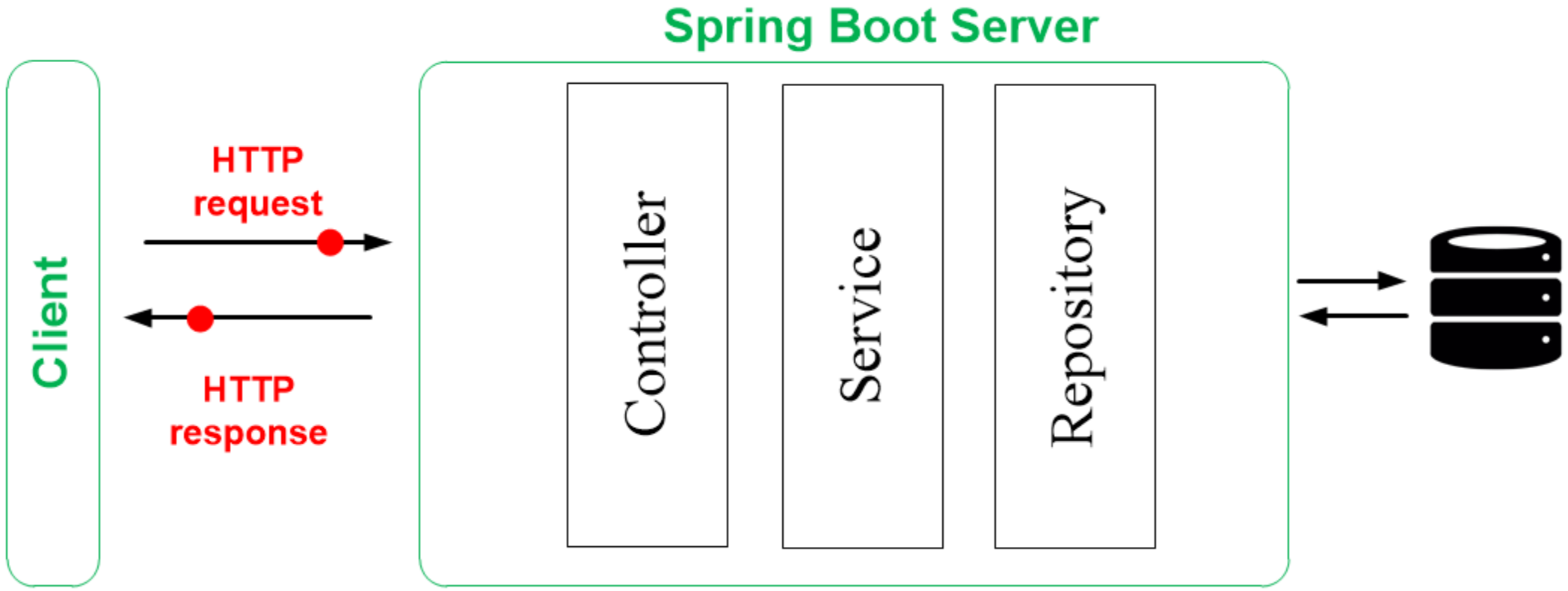
CRUD operations are the **four basic** operations of persistent storage.

Controller-Service-Repository pattern

REST Architecture

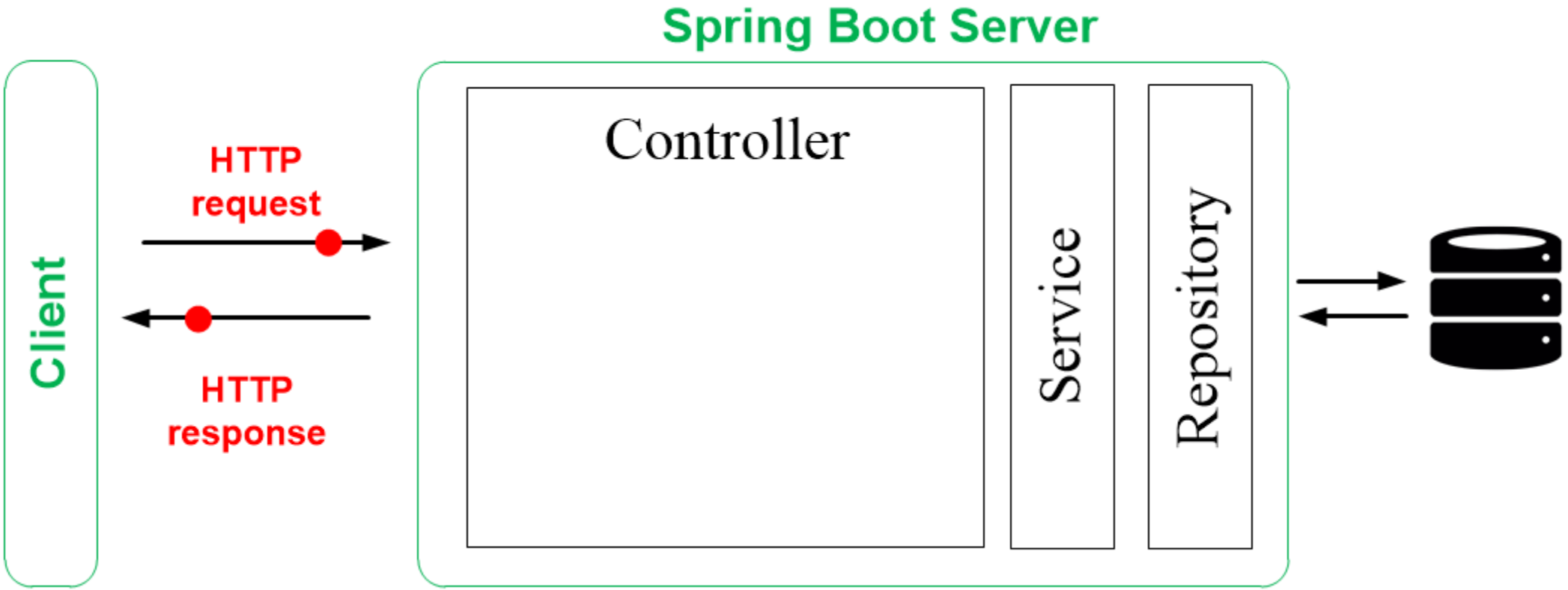


REST Architecture – CSR pattern



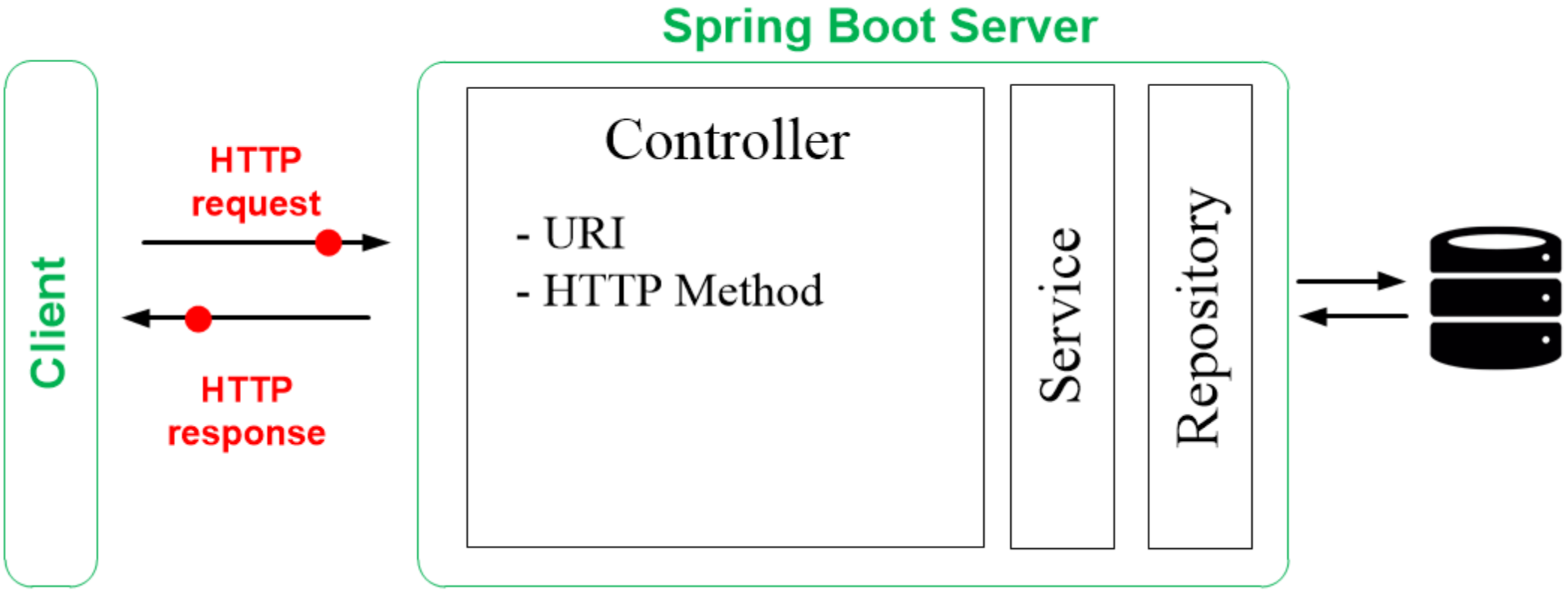
Controller-Service-Repository pattern

REST Architecture - Controller



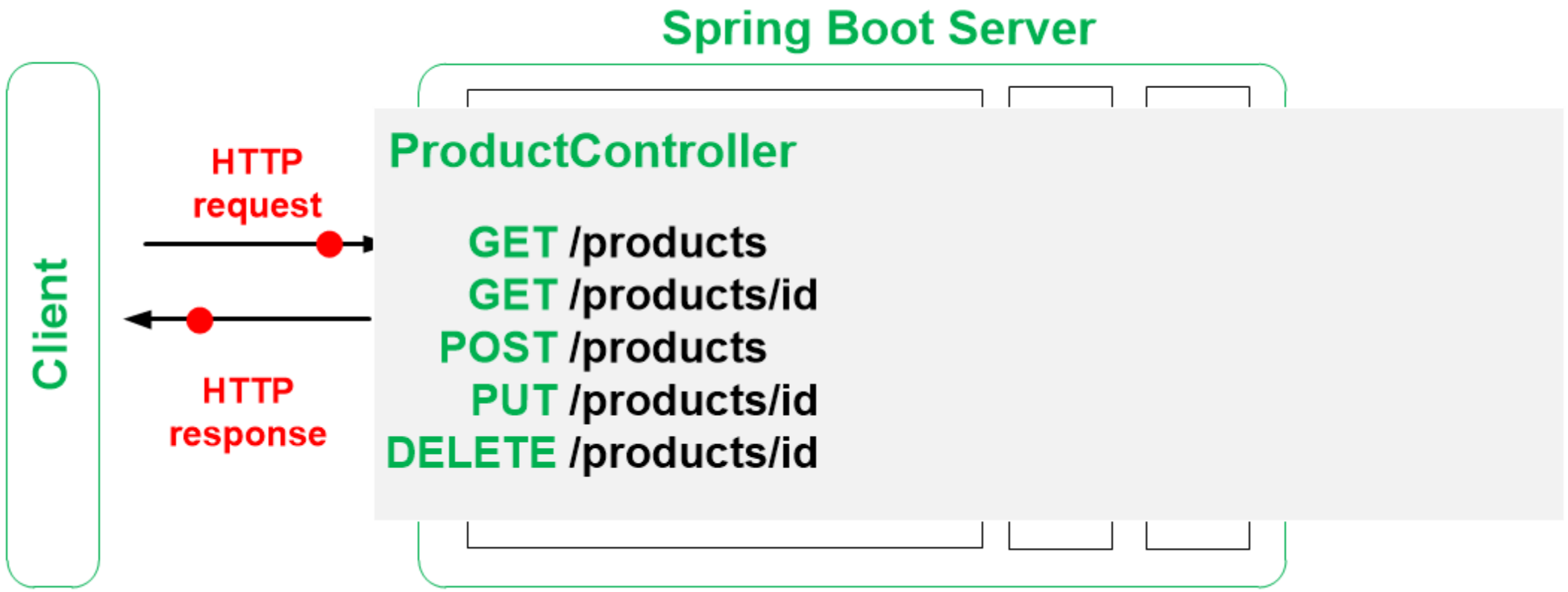
A Controller maps ***clients requests*** to corresponding ***services/functionalities*** (Java methods).

REST Architecture - Controller



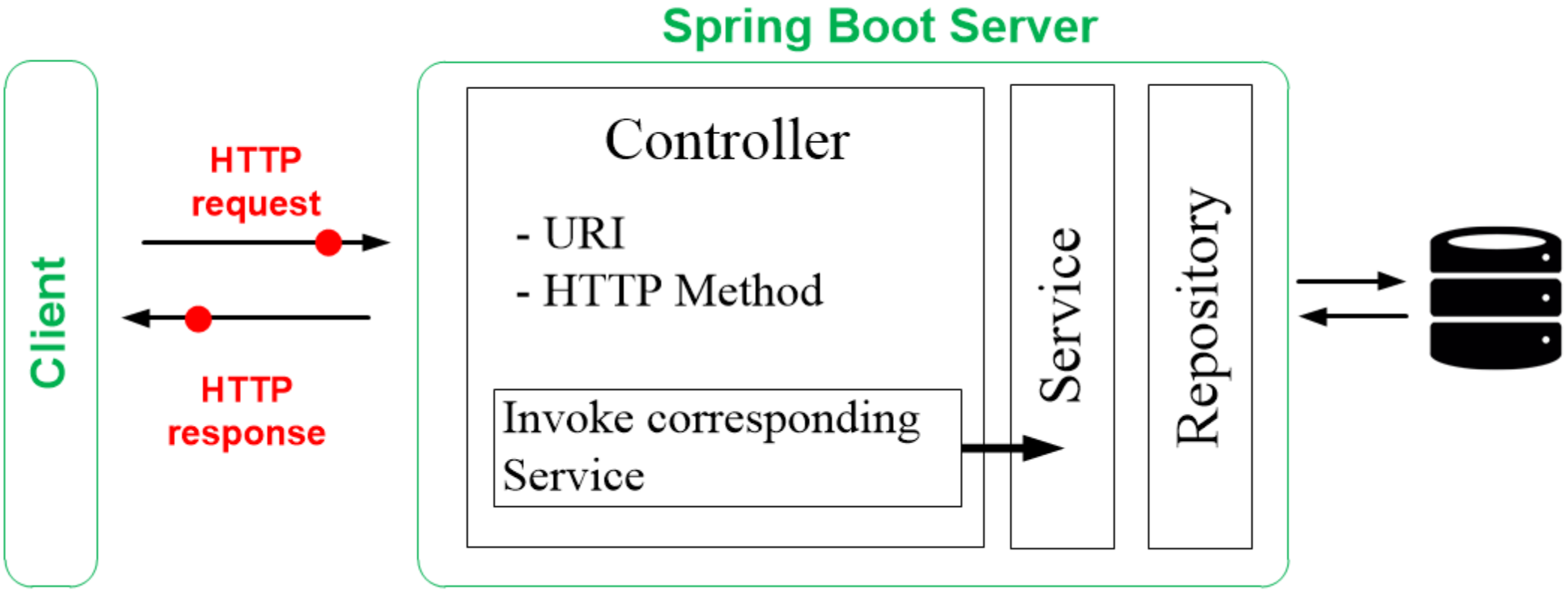
A Controller maps ***clients requests*** to corresponding ***services/functionalities*** (Java methods).

REST Architecture - Controller



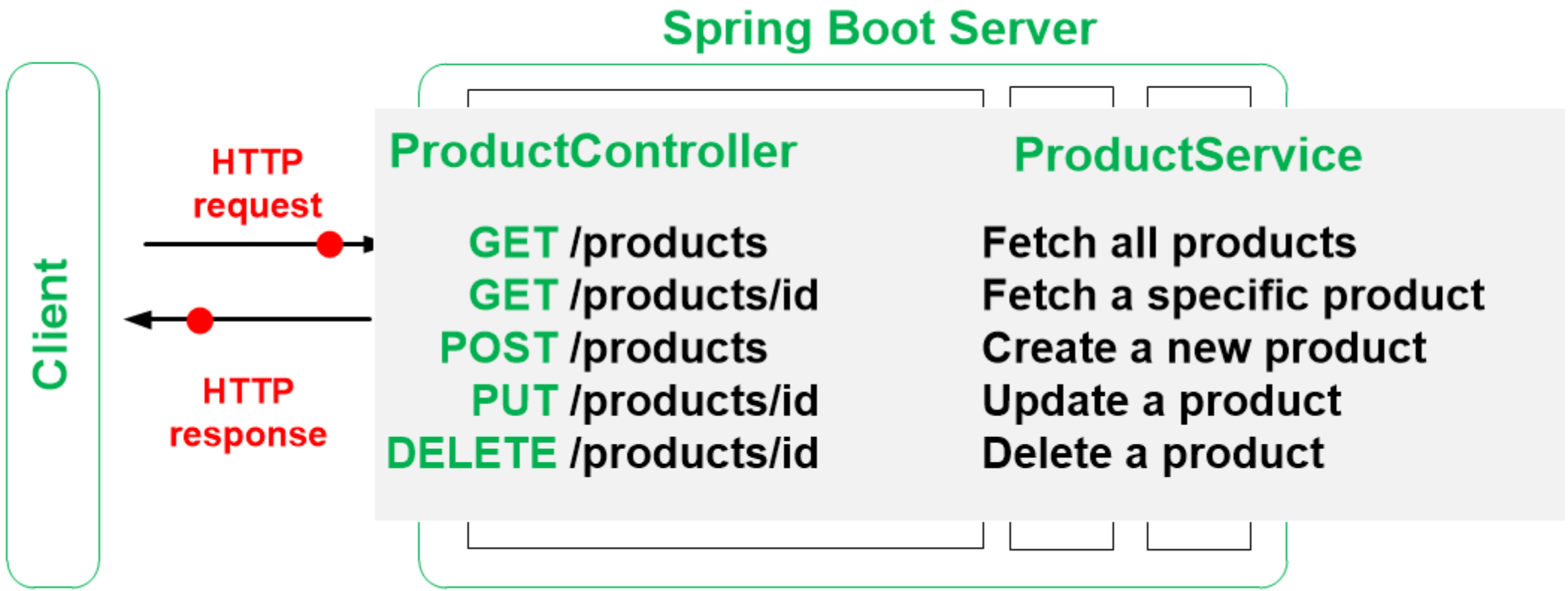
A Controller maps ***clients requests*** to corresponding ***services/functionalities*** (Java methods).

REST Architecture - Controller



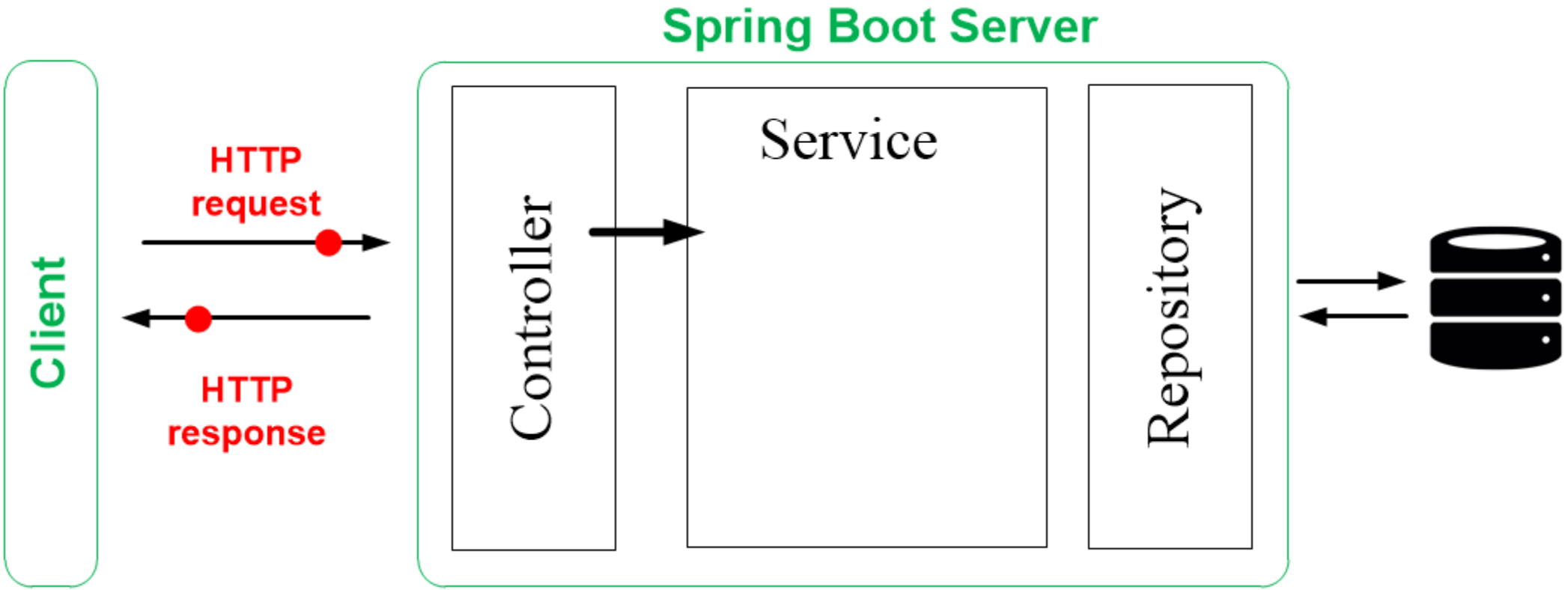
A Controller maps ***clients requests*** to corresponding ***services/functionalities*** (Java methods).

REST Architecture - Controller



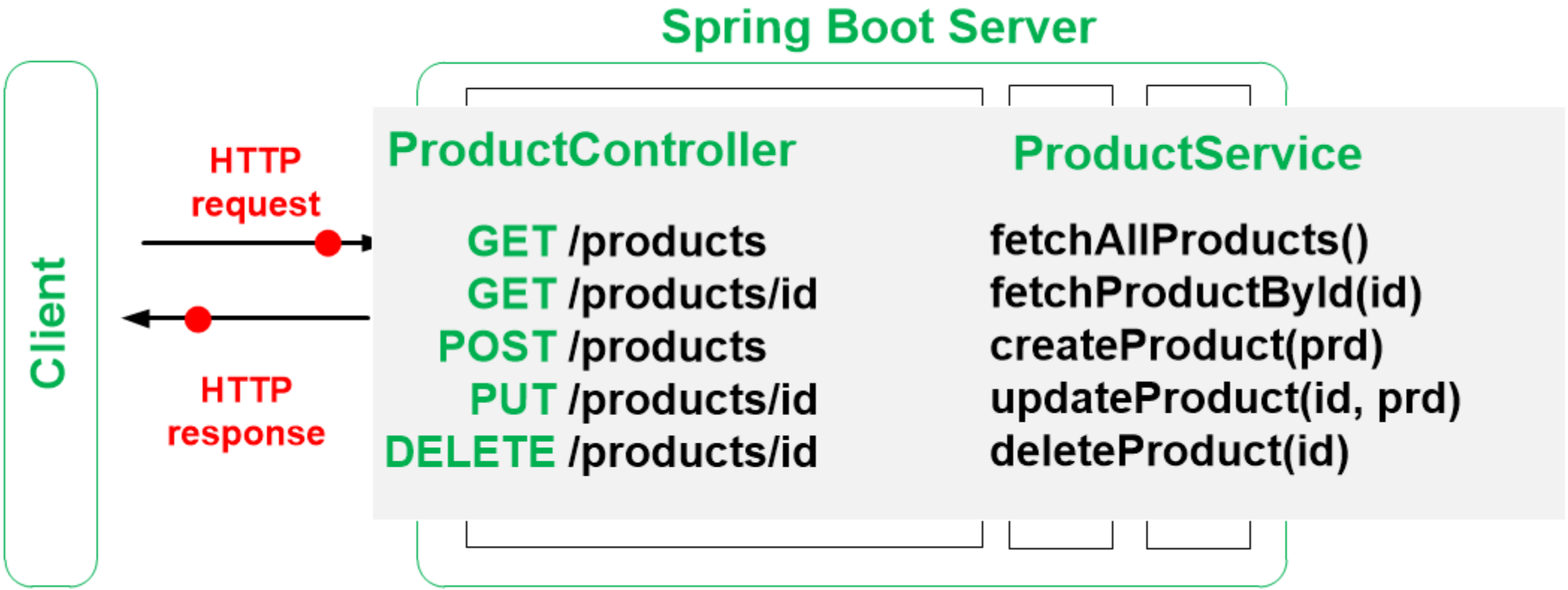
A Controller maps ***clients requests*** to corresponding ***services/functionalities*** (Java methods).

REST Architecture - Service



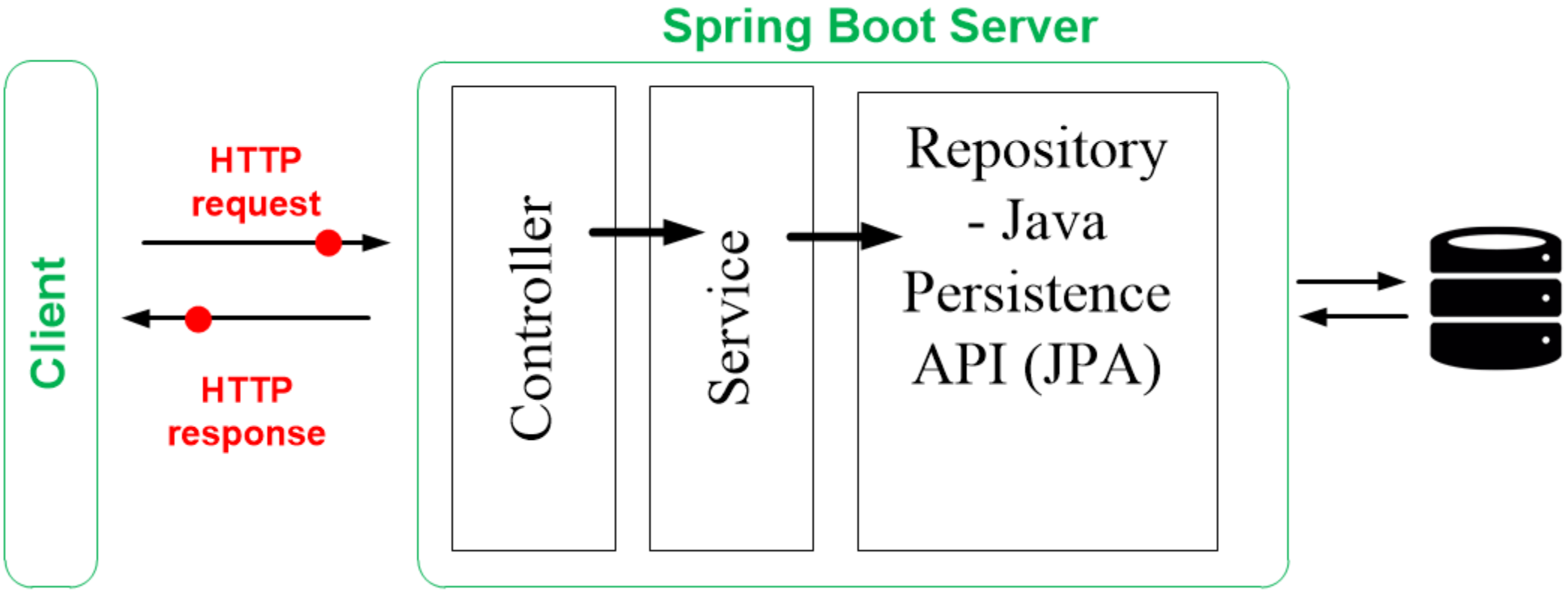
A Service (a method) implements the business logic underlining the service

REST Architecture - Service



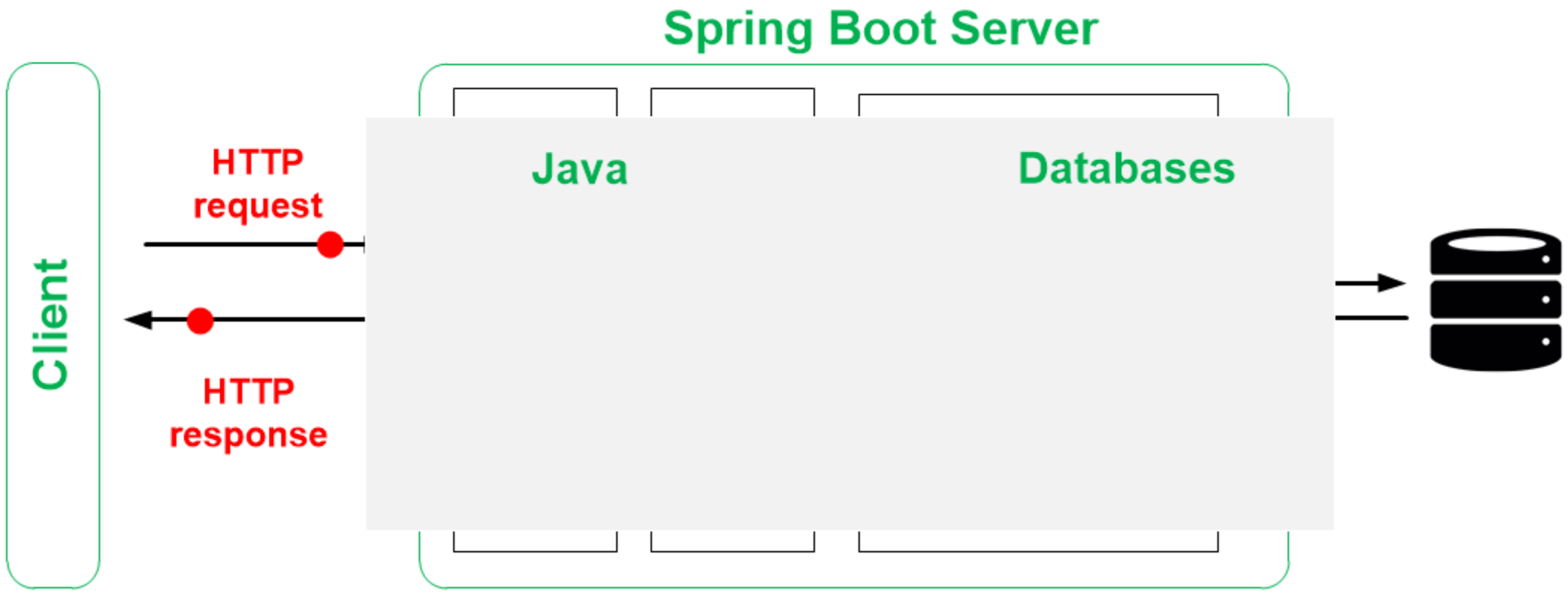
A Service (a method) implements the business logic underlining the service

REST Architecture - Repository



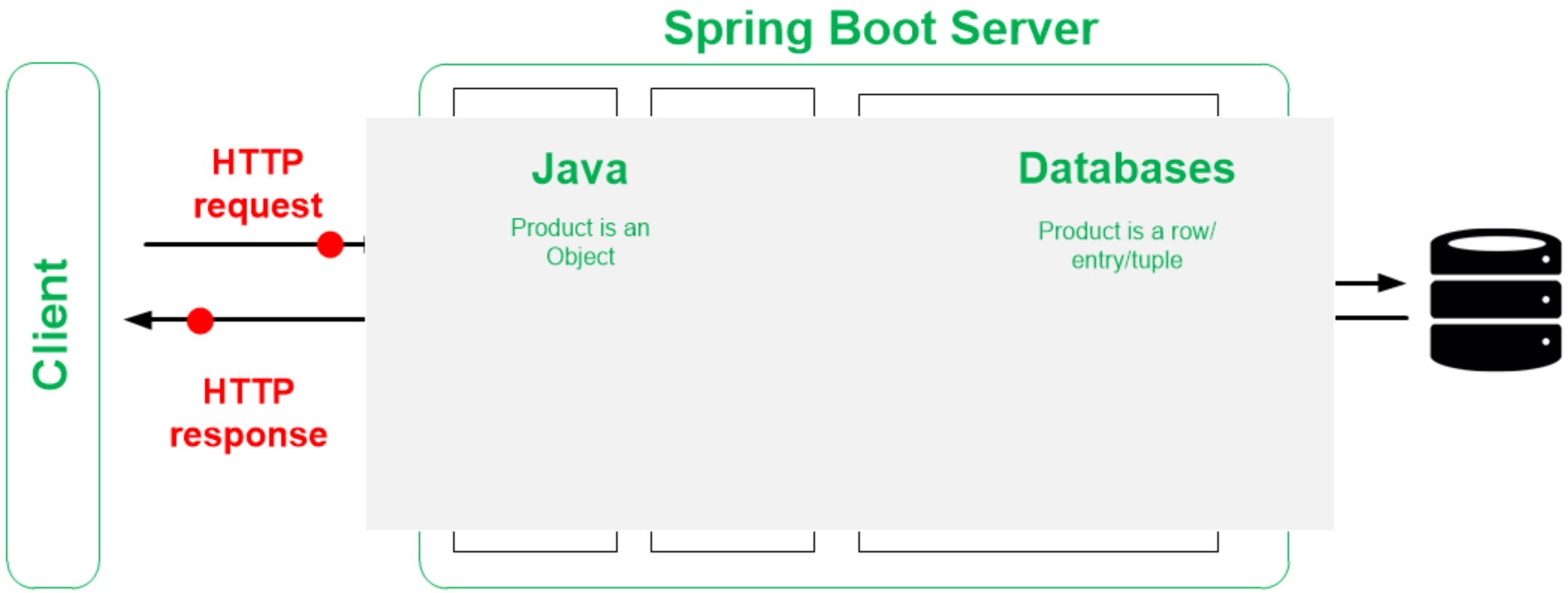
A Repository is responsible for retrieving, storing, updating and deleting some set of data.

REST Architecture - Repository



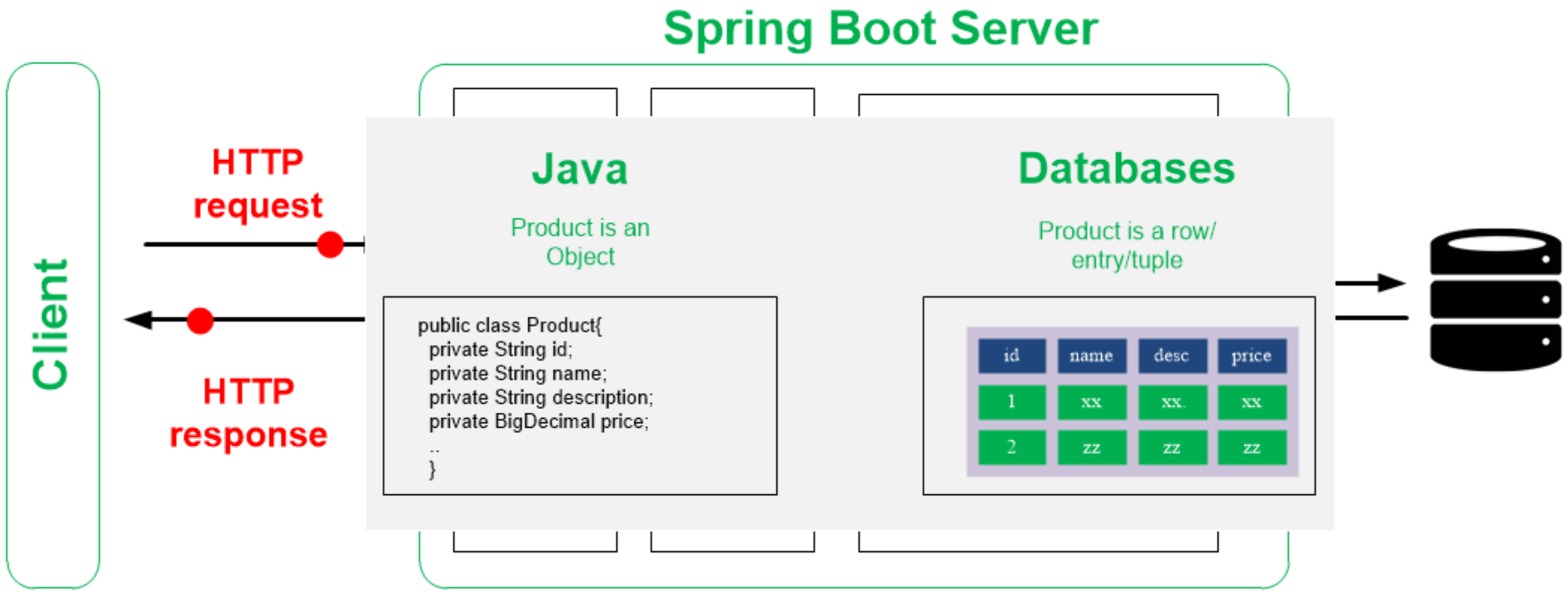
Java relies on the object-orient paradigm, and ***databases*** store data as rows and columns in a series of tables

REST Architecture - Repository



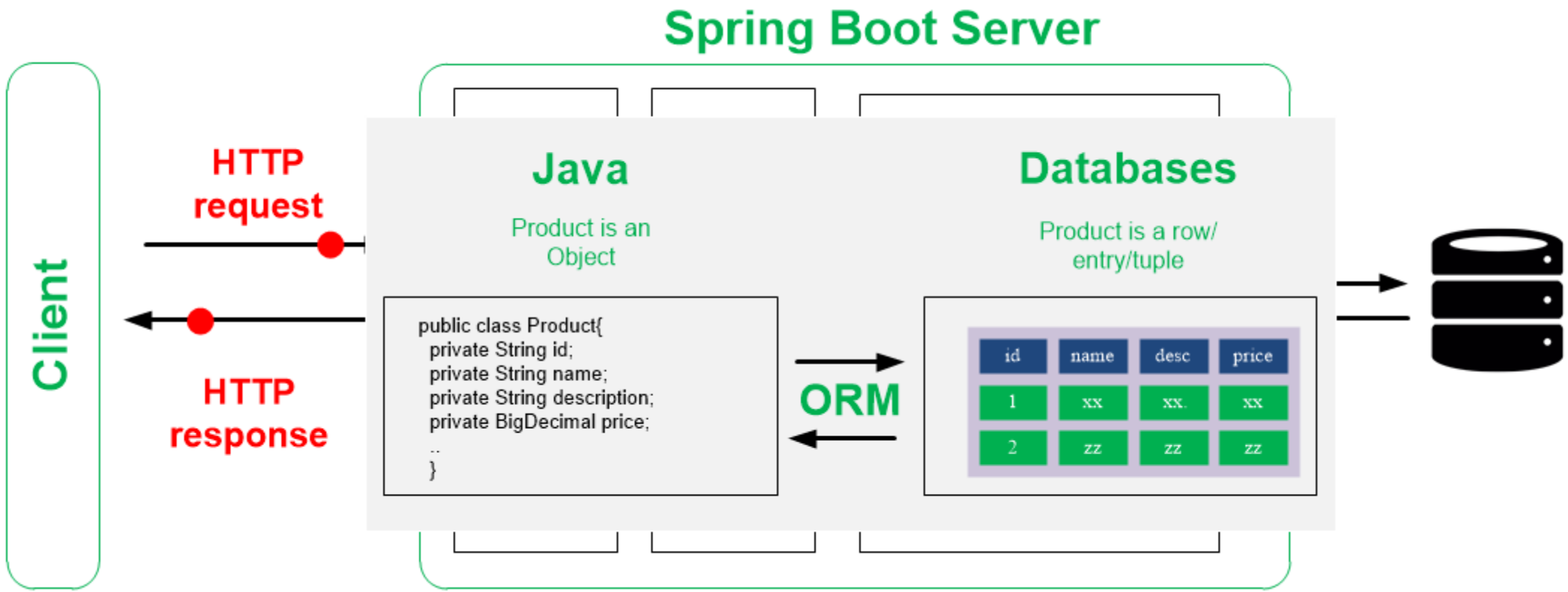
Java relies on the object-orient paradigm, and ***databases*** store data as rows and columns in a series of tables

REST Architecture - Repository



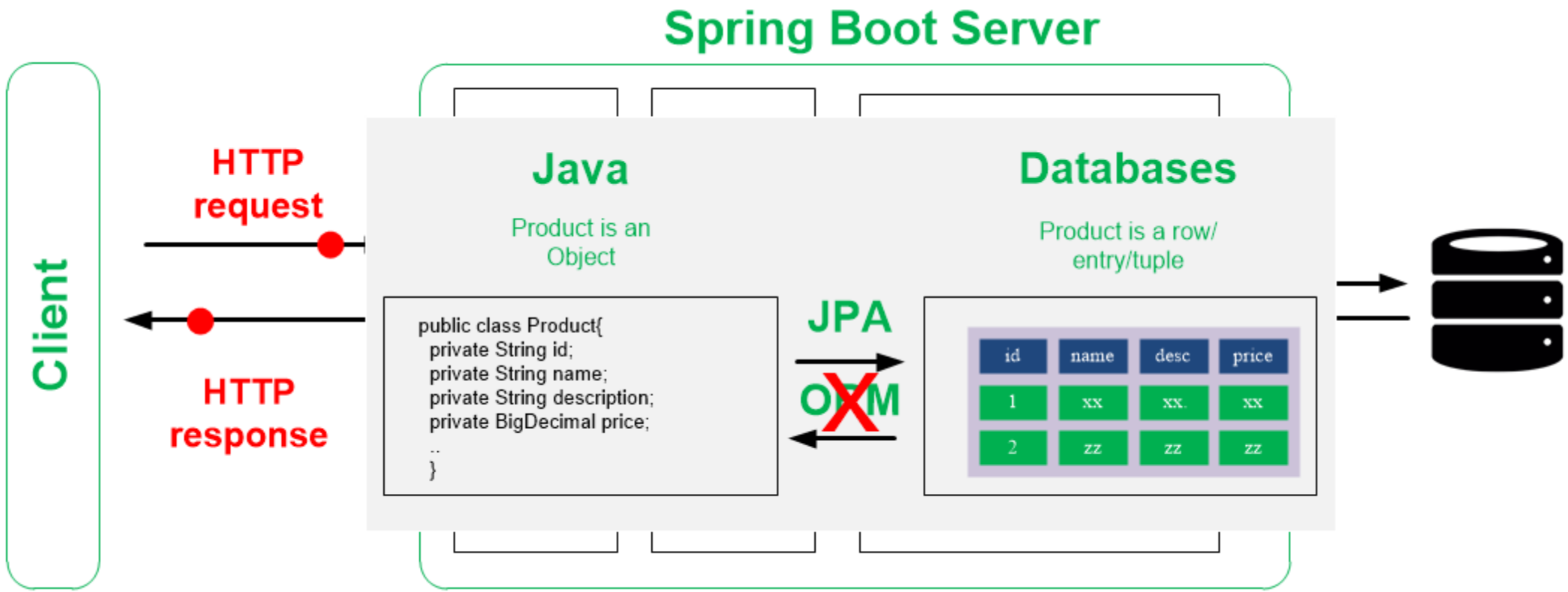
Java relies on the object-orient paradigm, and ***databases*** store data as rows and columns in a series of tables

REST Architecture - Repository



Object–relational mapping (ORM) is used for converting data between incompatible type systems using object-oriented programming languages.

REST Architecture - Repository



JPA provides Java developers with an ***object/relational mapping*** facility for managing relational data in Java applications.

REST Architecture - Repository

Spring Boot Server

ProductRepository

Public **interface** class ProductsRepository extends **CrudRepository** {...

@Autowired
Private ProductsRepository productsRepository

GET	/products	fetchAllProducts()	productsRepository.findAll()
GET	/products/id	fetchProductById(id)	productsRepository.findById(id)
POST	/products	createProduct(prd)	productsRepository.save(prd)
PUT	/products/id	updateProduct(id,prd)	productsRepository.save(prd)
DELETE	/products/id	deleteProduct(id)	productsRepository.delete(id)

JPA & ***CRUD*** Operations

The CSR pattern - practical example

The CSR pattern - practical example

In this example, we have two entities:

- Product
- Order

esi.~.products

Product (Entity)

ProductController

ProductService

ProductRepository

The CSR pattern - practical example

In this example, we have two entities:

- Product
- Order

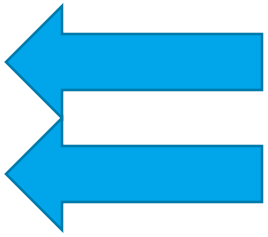
esi.~.products

Product (Entity)

ProductController

ProductService

ProductRepository



The CSR pattern - practical example

In this example, we have two entities:

- Product
- Order

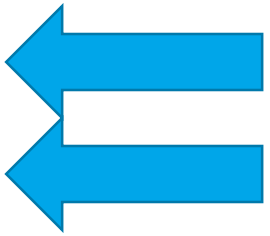
esi.~.products

Product (Entity)

ProductController

ProductService

ProductRepository



The CSR pattern - practical example

Spring Boot Server

ProductRepository

Public **interface** class ProductsRepository extends **CrudRepository** {...

@Autowired
Private ProductsRepository productsRepository

GET	/products	fetchAllProducts()	productsRepository.findAll()
GET	/products/id	fetchProductById(id)	productsRepository.findById(id)
POST	/products	createProduct(prd)	productsRepository.save(prd)
PUT	/products/id	updateProduct(id,prd)	productsRepository.save(prd)
DELETE	/products/id	deleteProduct(id)	productsRepository.delete(id)

JPA & ***CRUD*** Operations

The CSR pattern - practical example

```
public class Product{


    private String id;

    private String name;

    private String description;

    private BigDecimal price;

..
..
}
```

```
@Entity
@Table(name = "producttable")
public class Product{

    @Id
    private String id;
    @Column(name = “name")
    private String name;

    private String description;

    private BigDecimal price;

..
..
}
```

The CSR pattern - practical example

@Entity means that data contained in the class will be stored in database table

```
    private String id;

    private String name;

    private String description;

    private BigDecimal price;

..
..
}
```

```
@Entity
@Table(name = "producttable")
public class Product{

    @Id
    private String id;
    @Column(name = "name")
    private String name;

    private String description;

    private BigDecimal price;

..
..
}
```

The CSR pattern - practical example

@Entity means that data contained in the class will be stored in database table

@Table(name = "producttable") can be used if we want to give the table in the database different name. Otherwise, it will have the name of the entity by default

```
    private BigDecimal price;

..
..
}
```

```
@Entity
@Table(name = "producttable")
public class Product{

    @Id
    private String id;
    @Column(name = "name")
    private String name;

    private String description;

    private BigDecimal price;

..
..
}
```

The CSR pattern - practical example

@Entity means that data contained in the class will be stored in database table

@Table(name = "producttable") can be used if we want to give the table in the database different name. Otherwise, it will have the name of the entity by default

@Id indicates the member field below is the primary key of the current entity.

..
}

```
@Entity
@Table(name = "producttable")
public class Product{

    @Id
    private String id;
    @Column(name = "name")
    private String name;

    private String description;

    private BigDecimal price;

..
..
}
```

The CSR pattern - practical example

@Entity means that data contained in the class will be stored in database table

@Table(name = "producttable") can be used if we want to give the table in the database different name. Otherwise, it will have the name of the entity by default

@Id indicates the member field below is the primary key of the current entity.

@Column(name = "name") can be used to give a different name for a column in the database.

```
@Entity
@Table(name = "producttable")
public class Product{

    @Id
    private String id;
    @Column(name = "name")
    private String name;

    private String description;

    private BigDecimal price;

..
..
}
```

The CSR pattern - practical example

```
public class Pl
    private Str
    private Str
    private Str
..
..
}
```

```
@Entity
e")
)
ption;
```

```
@RestController
public class ProductController {

@GetMapping("/products")
    public List<Product> getAllProducts(){
     //return products;
     return productService.getAllProducts);
    }
..
..
}
```

The CSR pattern - practical example

```
public class Pl

    private Str

    private Str

    private Str

..
..
}
```

```
@Entity
e")


)


ption;
```

```
@RestController
public class ProductController {

@Autowired
private ProductService productService;


@GetMapping("/products")
    public List<Product> getAllProducts(){
     //return products;
     return productService.getAllProducts);
    }
..
..
}
```

@Autowired tells Spring to automatically create instance of ProductService and inject here because we are using its methods

The CSR pattern - practical example

```
public class Pl
    private Str
    private Str
    private Str
..
..
}
```

```
@Entity
able")
```

```
@RestController
public class PlantController {

@Autowi
private

@GetMap
    pub
     //
     re
    }
..
..
}
```

```
@Service //@Service tells spring that this is a service layer
public class ProductService {

        public   List<Product> getAllProducts(){
          List<Product> products =  new ArrayList<>();
          ProductRepository.findAll().forEach(products::add);
          return products;
        }
..
..
}
```

The CSR pattern - practical example

```
public class Pl
    private Str
    private Str
    private Str
..
..
}
```

```
@Entity
able")
```

```
@RestController
public class PlantController {

@Autowi
private
@GetMap
    pub
     //
     re
    }
..
..
}
```

```
@Service //@Service tells spring that this is a service layer
public class ProductService {

@Autowired
private ProductRepository productRepository;

        public   List<Product> getAllProducts(){
          List<Product> products =  new ArrayList<>();
          ProductRepository.findAll().forEach(products::add);
          return products;
        }
..
..
}
```

The CSR pattern - practical example

```
public class Pl
    private Str
    private Str
    private Str
..
..
}
```

```
@Entity
able")
```

```
@RestController
public class PlantController {

@Autowi
private

@GetMap
    pub
     //
     re
    }
..
..
}
```

```
@Service
public class ProductService {

@Autowired
private ProductRepository productRepository;

    public   List<Product> getAllProducts(){
      List<Product> products =  new ArrayList<>();
..
..
}
```

```
import org.springframework.data.repository.CrudRepository;

public interface ProductRepository extends
CrudRepository<Product, String>{
  // String is the type of primary key of the product class

}
}
```

The CSR pattern - practical example

```
public class Pl
    private Str
    private Str
    private Str
..
..
}
```

```
@Entity
able")
```

```
@RestController
public class PlantController {

@Autowi
private
@GetMap
    pub
     //
     re
    }
..
..
}
```

```
@Service
public class ProductService {

@Autowired
private ProductRepository productRepository;

    public   List<Product> getAllProducts(){
      List<Product> products =  new ArrayList<>();
..
..
}
```

```
import org.springframework.data.repository.CrudRepository;

public interface ProductRepository extends
CrudRepository<Product, String>{
  // String is the type of primary key of the product class
  // Spring will automatically implement the interface.
}
}
```

REST

Try it yourself …

Useful Tools

Postman is an API platform that helps developers to design, build, and test their APIs.

Thunder Client is a lightweight Rest API Client **Extension** for Visual Studio Code.

REST Client allows you to send HTTP request and view the response in Visual Studio Code directly.

REST Client: https://marketplace.visualstudio.com/items?itemName=humao.rest-client

Postgres

Do it yourself …

Postgres

Within the material of the practical session of **week 2**, you can find detailed information on how to install Postgres.

- Install **Postgres**,
- We will use the following **configurations** for the database during the practical session:
  - **username= postgres**
  - **password = postgres**
  - **database name = esi2023**

It is better to choose **Postgres** for both the **user name** and **password**, and create a database and name it **esi2023** before your practical session.

UNIVERSITY OF TARTU

Thank You
for your attention

Mohamad Gharib
mohamad.gharib@ut.ee

unitartu

tartuuniversity